Tours & Travel Mobile Application

Submitted by

Shahid Umer

(AUIC-22SG-BSCS-6732)

Musharaf

(AUIC-22SG-BSCS-7029)

Supervised by

Dr. Amjad Khan
Assistant Professor

A final year project report submitted in partial fulfillment of the requirement for the award of Bachelor of Science in Computer Science

Department of Computing

Abasyn University Islamabad Campus

April 2026

APPROVAL FOR SUBMISSION

This is to certify that the project report entitled **"Tours & Travel Mobile Application: A Flutter-Based Tourism Management System for Pakistan**" has been prepared by **Shahid Umer** (University Registration Number: **AUIC-22SG-BSCS-6732)** and **Musharaf** (University Registration Number: **AUIC-22SG-BSCS-7029)** in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science (BSCS) at Abasyn University Islamabad Campus. The report has been reviewed and is considered suitable for submission and evaluation according to departmental requirements.

The work presented in this report reflects the planning, design, development, and evaluation of a mobile application developed to support destination exploration, hotel booking, car rental, map-based tourism guidance, payment simulation, and user-generated service listings. The undersigned approve that the project is academically useful and technically adequate for final review.

Approval Committee:

Supervisor

Name

Designation _____

Institute _____

Signature _____

_____

External Examiner

Name

Designation _____

Institute _____

Signature _____

_____

Internal Examiner

Name

Designation _____

_____

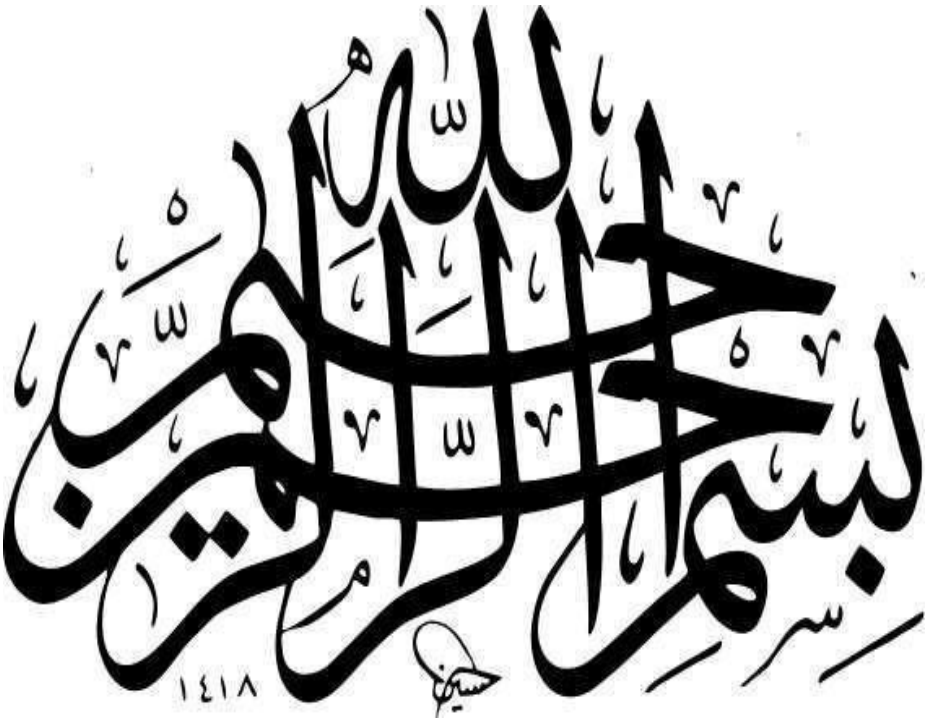
Institute _____

Signature _____

Head of Department:

Name _____

Signature _____



In the name of ALLAH, The Most Beneficent, The Most Merciful

DECLARATION OF ORGINALITY

We hereby declare that this project report entitled **"Tours & Travel Mobile Application"** is based on our original work except where due acknowledgement has been made through references, captions, or citations. The software artifact, documentation, and analysis included in this thesis were prepared specifically for the fulfillment of the final year project requirement and have not been submitted elsewhere for another degree or award.

We further undertake that if any part of this work is found to involve plagiarism or unauthorized copying, the university shall have the right to cancel the submission or take disciplinary action according to academic policy. All code, diagrams, screenshots, and explanations included in this thesis are either produced during this project or properly adapted from officially documented resources.

Signature**:**_____

Shahid Umer

Signature**:**_____

Musharaf

Submission and Copyrights

This project report is submitted to the Department of Computing at Abasyn University Islamabad Campus as partial fulfillment of the requirements for the award of the undergraduate degree. The authors grant the university permission to preserve the report for academic review, library record, and educational reference subject to institutional policy.

Shahid Umer
(AUIC-22SG-BSCS-6732)

Musharaf
(AUIC-22SG-BSCS-7029)

Dr. Amjad Khan (Supervisor)

Acknowledgment

We are truly thankful to the Almighty Allah the Most Beneficent and the Most Merciful for giving us strength, patience and steadiness to accomplish this final year project and this report. Every aspect of this work right from requirement analysis till the designing of the application and the report became easy with His mercy. We express our profound sense of indebtedness and sincere gratitude to our project supervisor **Dr. Amjad Khan**, Assistant Professor for his invaluable advice, guidance and positive criticism all the way through the process. It was indeed His ideas and guidelines which transformed a mere idea to a well designed mobile application with proper scope, modular structure and an understandable report. We are also grateful to the faculty of the department of Computing, Abasyn University Islamabad Campus for their consistent efforts in the form of lecture, lab session, guidance and technical expertise provided, which gave us confidence and necessary inputs for our real world software application. The everlasting love and support of our parents, siblings and friends has been our major encouragement. It was due to their care and affection that we were never demotivated throughout our work in difficult periods of testing, documentation, presentations and debugging which involved extended hours of development and modifications. Lastly we would also like to thank each one who is directly and indirectly involved in this project by the name, as a tester, as a user and provider of suggestions so as to improve the usability and practical utility of our application and clarity of our report.

ABSTRACT

Pakistan has strong tourism potential, from northern valleys, lakes, and mountain routes to historic, cultural, and urban destinations. However, many tourists still depend on scattered sources for destination information, hotel selection, transport planning, booking confirmation, and payment guidance. This final year project presents a Tours and Travel mobile application designed to combine these activities into one organized, user-friendly platform. The system is developed with Flutter for a consistent Android experience and uses Firebase services for authentication, cloud data storage, and profile-related support, while local fallback logic helps the prototype remain usable during demonstration. The application allows users to register or log in, explore destinations, view tour packages, select hotels, book cars, review booking summaries, simulate payment options, check booking history, manage profile details, use map-based destination support, and add hotel or car listings. The project follows a layered architecture in which interface screens, service classes, models, local storage, and Firebase integration are separated to improve maintainability and clarity. Requirement analysis, feasibility study, system design diagrams, implementation details, and manual testing are used to evaluate the application as an academic prototype. The results show that the system can support common travel-planning workflows through clear screens, reusable components, and structured data handling. Although the current version has limitations such as simulated payments, incomplete automated testing, and partial cloud integration in some flows, it establishes a practical foundation for a more complete travel booking platform. Future improvements may include real payment gateway integration, stronger search and recommendation features, admin dashboards, improved security rules, and broader automated testing. Overall, the project demonstrates how mobile and cloud technologies can be combined to simplify tourism planning and provide a more connected digital travel experience for users in Pakistan.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

Abbreviations	Acronyms
OTP	One Time Password
EMS	Expedition Management System
POS	Point of Sale
SMS	Short Message Service
UML	Unified Modeling Language

LIST OF TABLES

LIST OF FIGURES

Chapter 1
1 Introduction

1.1 Introduction

Technology has already transformed how services are found, searched and delivered in almost all sectors. Travel, more than anything else, is arguably the sector that has seen most change, since people now expect routes, options for accommodation, digital profile, confirmations, help tools readily available on their mobile phone. Mobile applications with these features no longer considered an extra. It has been essential in services provided and trip organization.

The Tours & Travel Mobile Application project attempts to address this change. It introduces a mobile application incorporating all the more common functions a traveler may require in the scope of domestic tourism: finding out about places to visit, comparing traveling packages, choosing a hotel, hiring a vehicle, seeing locations on a map, confirming a booking using a payment module, and profile and listing management. This is more than just a UI demo as the project consists of an application designed with defined services, models and user-flow structure.

This application provides a great opportunity to experiment software engineering principles with an application of both booking logic and UI activities. This mobile application covers authentication mechanisms, cloud and local storing, designing of a navigation flow, developing reusable widgets, creating logical validation and utilizing device functionalities such as taking pictures and handling of locations and provides a good final year project since it links theory with practice.

1.2 Background of the Study

It has become evident of traveler's interest in Pakistan's tourism industry through recent increases in traveling at national and international level as well, due to better travel and connectivity access along with awareness on social media networks and growing interest on beautiful locations such as: Hunza, Skardu, Swat, Fairy Meadows, Naran and many more. Although, the travelers have access to information related to the destinations mentioned but complete travel planning is done through different channels such as; travel agencies' pages on social networks, hotel web portals, taxi booking applications, personal social media profiles of friends and colleagues and simple phone calls. Since these systems are scattered they lead to inefficiencies like; redundant work, re-entry of data and uncertain travel plans.

With an increase in user behavior with regard to mobile technology among travelers, it has become necessary to develop integrated systems for traveler planning suitable for a particular

locality. A traveler interested in exploring northern areas of the country would ideally seek information regarding locations, estimated costs, hotel accommodation, transportation means and assistance in navigation and maps in a single integrated system. Currently, separating each activity over several services makes planning extremely taxing on the traveler. A system which integrates different tasks under a single framework can alleviate this tension by structuring the typical tasks related to travel.

The application uses mobile as the solution and provides travelers with access to the services required by linking and consolidating all of them under a single, usable interface. This approach helps in linking and bridging the gap between what traveler needs and what is being provided to them while keeping the cultural aspects of Pakistan along with its geographies, costs etc.

1.3 Problem Statement

The travel planning process for local tourism in local markets consists of disjointed and semi-manual activities. Users often stumble upon destinations through social media, ask about tours through messaging services, research hotels on different websites and plan for transport services separately. This leads to slow down, reduced visibility over the total cost, inconsistent data and a larger risk of missed communication between customers and service providers.

The absence of a singular, user friendly mobile application encompassing tours, hotels, transport and travel support makes the process difficult for users and inefficient for service providers. A traveler may be interested in a certain destination but not be able to go from discovery to a confirmed booking in a seamless manner. Correspondingly, service providers lack an structured approach to manage and present offerings, or handle customer interactions effectively.

The fundamental problem that the Tours & Travel Mobile Application addresses is the fact that there is no tourism application on the market today that takes care of exploration, booking, navigation, profile management, as well as the possibility for expansion into one seamless mobile application product in use within Pakistan.

1.4 Objectives of the Study

The objective of this project is to design and construct a tourism application in Flutter that consolidates destination viewing, accommodation booking, car rental and support for travel, within a single and accessible application. The solution would demonstrate quality in project construction, as well as user friendliness through modular rather than independent screens and components.

- To design a single application for browsing and booking travel and destinations.

• To integrate authentication, booking, map and search features as well as user profile and listing information.

• To practice application of Firebase and Flutter within an actual software development project, as part of academic learning.

• To show evidence of modular design as well as reusable UI design components.

• To establish a scalable structure for potential future enhancements, for commercial or research purposes.

1.5 Scope of the Project

The project's scope comprises designing and building of a mobile application from the traveler's point of view in the tourism flow which includes authentication, viewing destinations, supporting of one-city tours and multi-city tours, hotel searching and booking, car renting, displaying on a map, simulating of payment, managing of tour history, user's profile management and user-created entries of hotels and cars.

The scope does not include actual payment gateway implementation, a real production launch, any recommendations algorithms or a full administrative dashboard or content moderations, these should be considered as next steps

1.6 Significance of the Project

The value of the project stems from transforming a prevalent real world problem into a defined mobile solution. It demonstrates how software engineering techniques can be applied to define user requirements, map them onto screen flows, link to server-side services and present a technically viable and socially beneficial product.

The project also provides an exemplar for multi-platform development when budget, device variety and development time are critical constraints. Flutter provides a powerful platform for designing a contemporary user interface on more than one platform and Firebase mitigates server-side complexities for a student team.

1.7 Research Questions

The following research questions drive the design of the project: "How will a mobile app diminish fragmentation within domestic trip planning?", "What modules create most utility when clustered for the tourism user?", "What design aspects can enhance demonstration resiliency for an educational project through the use of hybrid storage?", "How can the travel app be designed to allow first time users easy and quick access?".

These questions drive not just the design of the application, but also what criteria will measure success, beyond a simple completion, and thesis will measure flow clarity, modularity, simplicity and extensibility.

1.8 Report Organization

This report consists of 8 chapters. The first chapter describes the problem area and context, rationale and goals of the project, as well as its significance. The second chapter gives an overview of related technologies, prevalent travel application structures and the gap this project seeks to address. Chapter 3 focuses on requirements engineering and feasibility study.

Chapter 4 details system analysis and design models, including architecture, flow and schema design, and other diagrammatic representation of the system. The implementation description of individual modules is laid out in chapter 5. The sixth chapter focuses on validation and evaluation aspects of the developed system. Chapter 7 provides the summary of the achieved result, discusses the drawbacks and the deployment strategy of the system, while the concluding chapter, chapter 8 provides conclusions and future scope.

Chapter 2
2 Literature Review

2.1 Tourism Applications and Digital Service Aggregation

Contemporary tourism applications aspire to minimize planning effort by bringing together information, booking actions, and travel support into one channel. Large systems usually offer package deals, hotels, ratings, maps and marketing material, however these services have typically been optimized for worldwide travel and not for local country-specific patterns of usage. In Pakistan, consumers turn to less structured systems, such as personal agents, individual recommendations found in messaging groups or on social media posts, which lack the consistency of structured digital products.

A literature review on digital tourism services indicated that "the more valuable systems were those that shorten the path from awareness to action". In other words, consumers should go from desire to decision and from decision to final confirmation, without ever changing the underlying tool or application. This is how the application described herein was designed: all activities such as, exploring the city, finding the desired hotel, booking a vehicle, viewing maps and having access to one's personal past within the application will be integrated into a single mobile flow.

2.2 Cross-Platform Mobile Development with Flutter

Cross-platform development was chosen by many startups and academic projects since it cuts development costs but gives a consistent experience across the systems. Especially Flutter uses composition of UI from widgets, has expressive UI and enables fast prototyping. Having features like Hot Reload and a layered design along with strong community support is considered beneficial when a feature-rich application has to be developed within a limited time frame.

Flutter was chosen not just for speed, but also because of design expressiveness; the lists, grids, transitions, images management, form validations, bottom navigation, custom colored components, are essential components of the application. With the use of Flutter rendering, a visually consistent design experience can be given throughout which are quite essential on traveling apps, as design significantly matters to user confidence and trust.

2.3 Firebase as a Backend Service

Firebase is commonly used in student and startup projects because it offers authentication, cloud database and file-storage and analytics-driven services without demanding a full custom server side. In the case of a mobile-based project, it lightens the burden of having an infrastructure and allows focusing more quickly on the business flows and on the interface, delivering features step by step.

In the current project, it allows storing account details, profiles, some example data, and images are uploaded and stored. The solution also represents a good educational example: cloud services are very useful but when building for an educational or demo scenario the lack of fully complete setup or of connectivity requires fallbacks that complement, rather than substitute, the cloud services (e.g., Shared Preferences fallback service).

2.4 User Experience Design in Travel Applications

User experience in travel applications must achieve visual appeal as well as ease of use. Users are making crucial decisions about price, convenience, destinations, and reliability. The success of the application can be undermined by a cluttered interface and confusing navigation, even if its logic is sound. Travel applications that work have successful user interfaces using cards, imagery, progressive disclosure, clear pricing, and clearly defined actions.

This application has succeeded by organizing destinations and tour packages into distinct cards, providing clearly separated tabs for bookings and summaries, and by outlining all choices of payment and destination cleanly. Even animations, gradients, and image-heavy discovery modules are more than just superficial flair, but an important means of bringing attention to significant user choices, while ensuring an up to date, yet not overwhelming, user experience.

2.5 Location-Based Services and Tourism Support

Location aware functionality is of most value in tourism systems where it aids in orientation and in navigation, and in the discovery of landmarks. The inclusion of maps can assist in the discovery and re-assurance processes; even rudimentary tourist markers with descriptions enhance the utility of an application significantly and provide links between abstract package names and physical locations. This project uses Google Maps and geolocation services to reinforce the user's mental model of destinations. Tourist landmarks designated by the user can be shown on the map as markers; the user's location can also be displayed to assist in navigation. The application provides further travel support beyond booking by incorporating these services, in keeping with research that demonstrates the utility of context aware design

and places based representation for travel sites.

2.6 Payment Interfaces and Trust in Booking Systems

The payment process is arguably the part of the whole system which is most sensitive towards trust. It is important to design the payment screen properly in a student project where a live payment gateway may not be part of it. The payment module must show a summary of the booking, present known ways of payment and clearly indicate booking status.

The project does contain a payment simulation interface and offers multiple payment choices-card, JazzCash, EasyPaisa and COD, and while it does not make real transactions yet, it does replicate the behavior. The process also generates a confirmation of the booking immediately to support the real world integration with production payment APIs.

2.7 Comparison of Existing Systems

An analysis of existing systems clearly indicates that most of the existing solutions are either very niche or isolated. Hotel booking applications are ideal for hotels, but may not cover route planning or local attractions. Mobility applications efficiently manage travel, but generally do not allow tourists to view a package. Websites belonging to a travel agent may provide details, but they do not present application level functionality and long term profile use.

Our application acts as a broad domestic travel solution rather than a specialized one. The main strength of our system is bringing together a number of medium size yet deeply integrated components. This comparison helps us to see the research gap and supports why integration as an objective itself is itself an innovation.

Table 1 2.1: Comparative View of Existing Travel-Related Systems

Abbreviation	Full Form
API	Application Programming Interface
BaaS	Backend as a Service
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
GPS	Global Positioning System
JSON	JavaScript Object Notation
OTP	One-Time Password
PKR	Pakistani Rupee
SDK	Software Development Kit
UI	User Interface

UML	Unified Modeling Language
UX	User Experience
URL	Uniform Resource Locator
MBaaS	Mobile Backend as a Service

2.8 Research Gap

Based on the review of the literature and market scanning, there is an obvious gap between the broad generic travel apps and localized, useful and context-driven travel-related application in Pakistan which is inspired by the use of Pakistan based travel user journey. A travel app that offers combined experience of destination browsing, package showcase, hotel search and booking, consideration of the local transaction trends, intra-city and inter-city vehicle reservation and user based business listing under a mobile based application should be designed and implemented.

This project attempts to fulfill this gap by presenting a proof-of-concept travel based application that is module oriented, extensible and does not propose a finished product in commercial market but a structured development at undergraduate student level with the plan, development and assessment of a tourism application.

Chapter3
3 Requirement Engineering and Feasibility Study

3.1 Stakeholder Identification

The primary users of the application will be: Travelers, tourist prospective users, owners of hotel and vehicle, the evaluators/supervisor of the project, and the system maintainers. Travelers will be the main user-facing system users, since they are interacting with the destination cards, the screens of booking, the map, and history modules. Hotel and vehicle owners will be second tier users as they are using the listing system which allows registering services in the application itself.

Academic evaluators/supervisor are also user of the application as the system should be demonstrable, under stable, and well structured technically to allow evaluation. Therefore, clean design, easily understandable document and smooth demonstrations will have to be present. System maintainers (whether new developers or owners) will need well separated service and module structures, as well as a clear names for extendibility.

3.2 Functional Requirements

These represent what the user wants the system to do. The user has to create an account, login using social sign in and normal authentication and access services. The system must provide the user with the ability to view destinations and tour packages, search and select hotels, book city-to-city car rides, use the interactive map and the ability to access the payment interface, receive booking confirmation, book history view and profile edit. Users are also supposed to add new hotel and car listing themselves, thus requiring the support for new booking forms and data submission through the system. Every functional requirement, which are specified above, is linked to a specific service/screen.

Table 2 3.1: Core Functional Requirements

Requirement ID	Description	Primary Module
FR-01	User registration and login	Authentication
FR-02	Destination browsing	Home and Discovery
FR-03	Multi-city tour browsing	Tour Packages
FR-04	Hotel search and selection	Hotel Booking
FR-05	City-to-city car booking	Car Rental
FR-06	Map and tourist marker	Maps
FR-06	exploration	Maps
FR-07	Payment and confirmation flow	Payment
FR-08	Booking history view	Booking History
FR-09	Profile update and image upload	Profile
FR-10	User-created hotel and car listings	User Listings

3.1 Non-Functional Requirements

These specify the quality attributes of the system. For this project ease of use, system responsiveness, modular structure, clarity of interface and acceptable behavior under partial setup are of prime concern. Users should find the app intuitively understandable without prior instruction, and the presented traveling options should be easy to read.

Easy maintenance is a key consideration, because the system comprises several modules which depend on one another. Naming, service bounds, reusable widgets and single constants become key architectural decisions. Also of some use is security, mostly in the realm of authentication and profiles which justifies Firebase on the server side for official support once installed.

Table 3 3.2: Non-Functional Requirements

Category	Expectation
Usability	Clear, mobile-friendly, and easy-to-understand screens
Performance	Smooth enough for demonstration on typical student devices
Maintainability	Modular code structure with reusable utilities
Reliability	Reasonable fallback behavior where cloud dependencies are incomplete
Scalability	Architecture should support future backend enhancement
Security	Use authenticated access where required and validate user input

3.3 Hardware and Software Requirements

A development machine is necessary for this project, on which the Flutter toolchain, emulators for Android devices, as well as a development environment such as Android Studio or Visual Studio Code can be run. For software components we need the Flutter SDK, Dart SDK, Android tool chain and files of Firebase settings in order to work efficiently. For the maps functionality, a valid API key is needed.

Looking at it from the usage point of view, this app is focused on mobile platforms. Although there are a web, desktop and platform scaffolding generated from the project using Flutter, as we do not work on them, so there is a way to make the app running on all of them.

Table 4 3.3: Hardware and Software Requirements

Type	Requirement
Development Hardware	Laptop or desktop capable of running Flutter tooling
Development Software	Flutter SDK, Dart SDK, Android Studio or VS Code

Target Platform	Android mobile device or emulator
Backend Support	Firebase project configuration
Maps Support	Google Maps API key
Optional Assets	Destination images and test profile images

3.4 Feasibility Study

The technical feasibility is quite good, as Flutter and Firebase can together be used to fulfill user authentication, cloud storage, mobile UI and location functionality, with minimum amount of back-end complexity. The domain problem can also be easily broken down into manageable modules, the possible user flows are; logging in/creating an account, discovering, booking, payment, managing profile.

The economic feasibility is also good, due to decrease in cost and development time thanks to the technologies selected. The operational feasibility is also very good, as most users are already accustomed to standard smartphone paradigms such as; log-in screens, cards, maps, lists, confirm-screens, etc.

3.5 Risk Assessment

Some project risks are in place that include inadequate setup of Firebase, erratic connectivity of the internet during demonstration, low test coverage, and risk of inconsistent states in each of the modules if the booking logic is not centralized. These are critical because even a well developed and visually appealing app could have issues with demonstrations if not properly implemented with environment assumptions considered.

The project addresses some of these risks through fallback login, local persistence of listings, and in-memory confirmation for bookings. None of these entirely eliminate all risks; however they address risks, make the app more robust with student demonstrations and make it more forgiving with server availability.

Table 5 3.3: Hardware and Software Requirements

Risk	Impact	Mitigation
Firebase configuration errors	Authentication or data issues	Fallback auth and careful environment setup
Incomplete automated tests	Lower confidence in regression behavior	Manual flow testing and future test expansion
Connectivity limitations	Cloud features may fail during	Local persistence and seed data
Risk	demo	approaches
Payment gateway absence	No real transaction capability	Use simulation with future integration plan
State inconsistency between modules	Unexpected booking visibility problems	Centralized service and store logic

3.6 Requirement Traceability

Requirement traceability allows us to ensure that all user requirements are implemented in our artifacts. In this case we have shown this through a trace to the screen, service, data model and test case. This is important for keeping track of the document and helping us later if changes need to be made. An example of a requirement trace for 'The user should be able to register their own hotel listing' would be a mapping to the Add Hotel Screen, User Listings Service, User Hotel data model, Shared Preferences, and specifically selected manual test cases. Likewise there is a trace for a user being able to upload a profile image, display of maps, and display of past bookings.

Table 6 3.5: Requirement Traceability Matrix

Requirement	Screen/Module	Service	Validation Focus
FR-01	Login / Signup	Local Auth Service	Credential flow and routing
FR-04	Hotel Search / Hotel Selection	Data base Service	Search and selection display
FR-05	Car Booking	Local Booking Store	Route, price, and booking visibility
FR-07	Payment Screen	Booking Service / Local Booking Store	Confirmation behavior
FR-09	Profile Screen	Auth Service / Firebase Storage	Profile update and image upload
FR-10	Add Hotel / Add Car / My Listings	User Listings Service	Local persistence and tab display

3.7 Summary

The requirements engineering phase is what is needed for the rest of the system design. This step provides clarity of the stakeholders, their required tasks, required quality characteristics and the associated risks. This specification is then ready for use in architecture planning and system design at module level.

Chapter 4
4 System Analysis and Design

4.1 Proposed System Architecture

This proposed design adopts a layered architecture where the presentation layer is segregated from the business layer and data layer. This layer mainly consists of screens, reusable widgets and cards. Just below that are the services that manages authentication, access to database, book flow and local storage and other utility elements such as theme setup, validators, animations, constants, extensions which minimize code repetition and create consistency throughout the app.

This is an ideal architecture for a student project as it gives both good conceptual structure and easy implementation. It doesn't create excessive over-engineering while ensuring Separation of Concern principles. The modular design makes it easier to present and test modules independently, as well as provide a more organized base to extend upon for the future development.

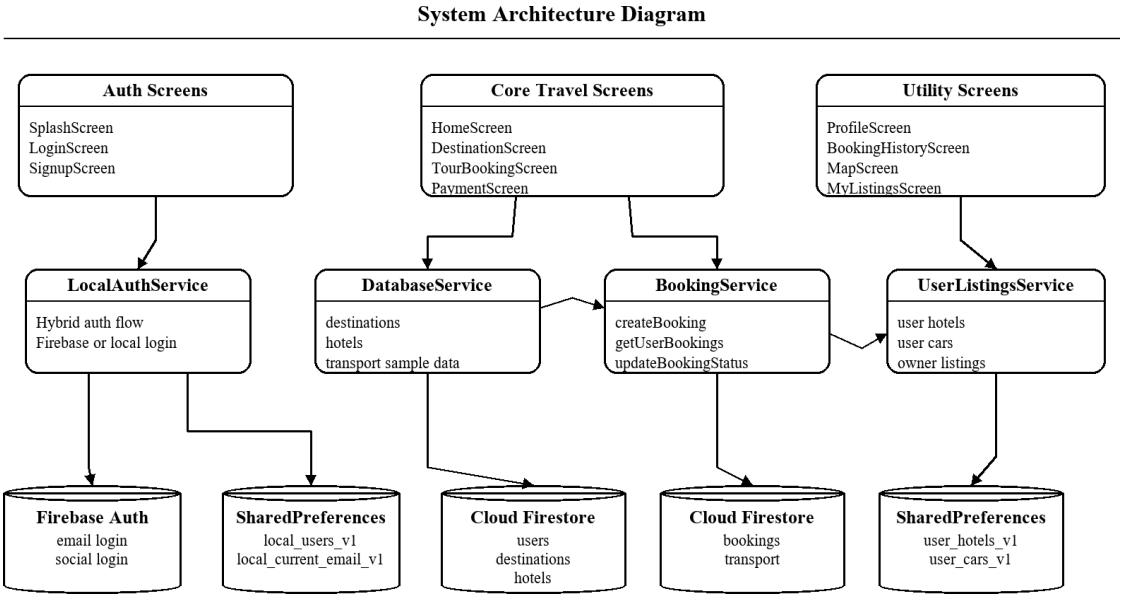


Figure 1 4.1: Proposed System Architecture Diagram

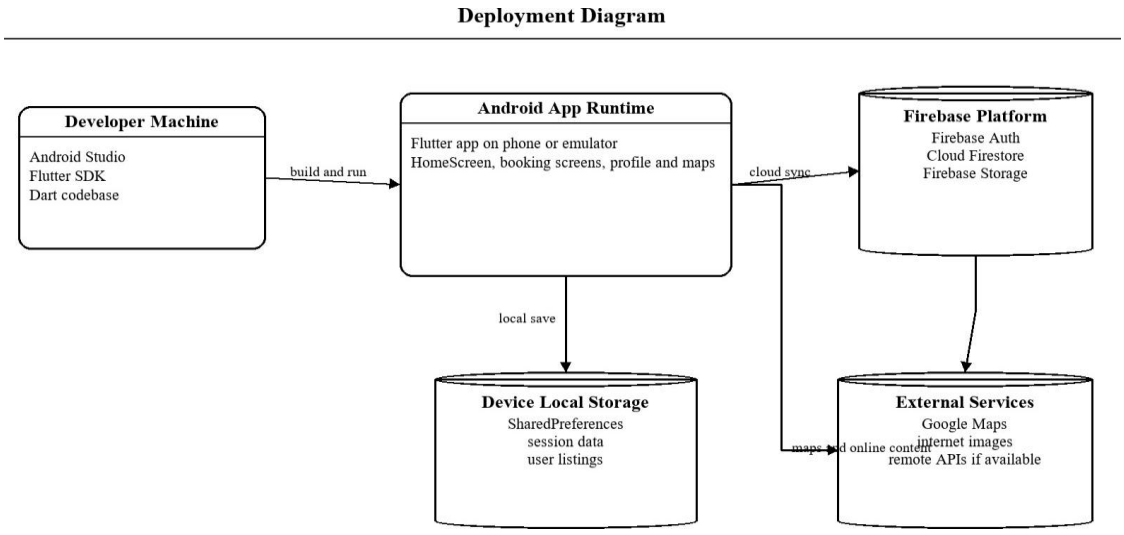


Figure 2 4.12: Deployment Diagram

Table 7 4.1: Proposed Technology Stack

Layer	Technology
Frontend	Flutter and Dart
Authentication	Firebase Auth and Local Auth Service fallback
Cloud Database	Cloud Firestore
File Storage	Firebase Storage
Local Persistence	Shared Preferences and in-memory stores
Maps and Location	Google Maps Flutter and Geolocator
UI Support	Google Fonts, animations, cached image tools

4.2 Use Case Perspective

From a use case perspective, the system is built around a user (registered or guest) using a mobile application to interface with travel related services. The main use cases are, in essence, "creating an account," "logging in," "viewing a destination," "selecting a tour," "searching for a hotel, "booking a rental car," "viewing the map" and "configuring preferences," confirming booking". There are additional use cases in regards to registering a personal listing of hotels and rental car.

The Use Case perspective is critical because it moves analysis from simply screens to user intents. Instead of discussing the system only in terms of interface components it questions what actions are valuable to the user and what ordered steps the system has to support.

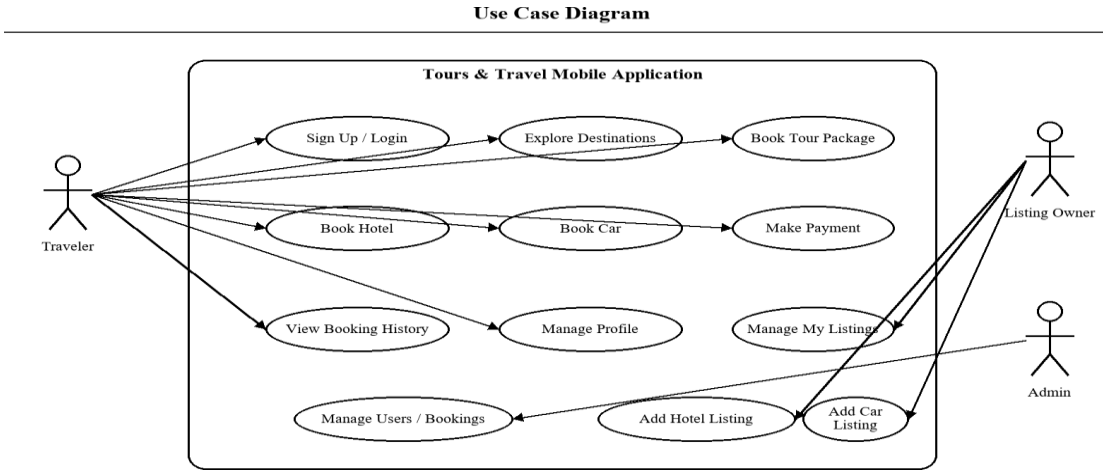


Figure 3 4.2: Use Case Diagram

4.3 Navigation Design

Navigation design is screen based and imperative. On startup the application leads to authentication or directly to the homepage upon authentication. The bottom navigation and card-based action flows allow access to tours, history, profile, map and listings from the homepage. The importance of navigation design in the context of a travel application, users flip-flop between finding information and confirming selection, cannot be understated. In this case, I have focused on observable actions, familiar navigations and summary screens to avoid confusion. Every booking flow seeks to maintain context, allowing the user to recognize what it is that he has chosen and why the system requires the next input.

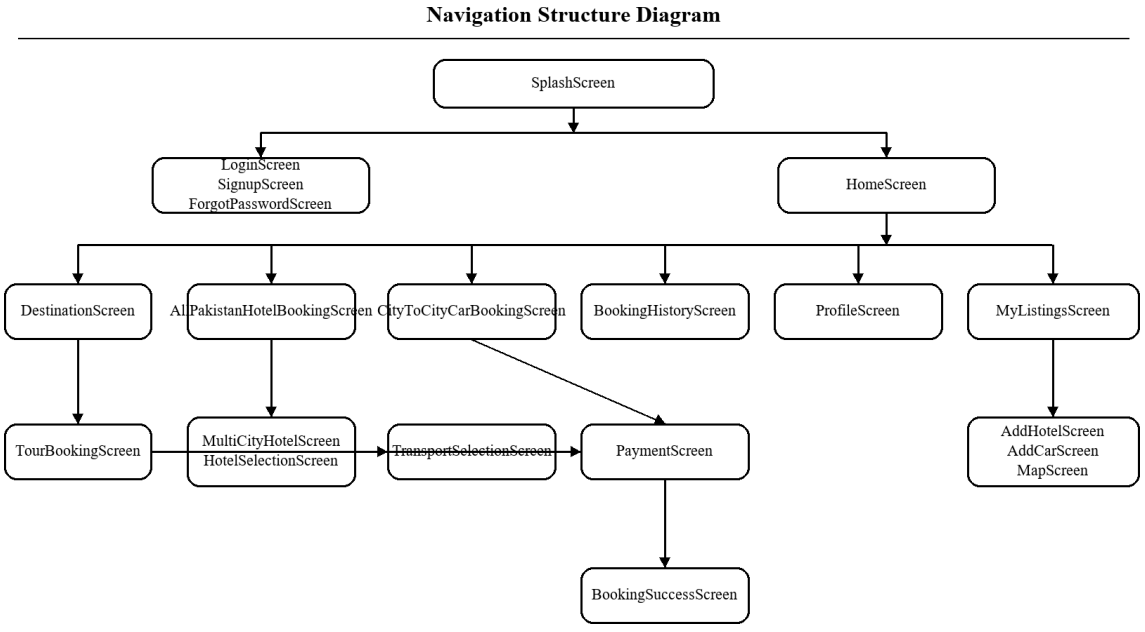


Figure 44.11: Navigation Structure Diagram

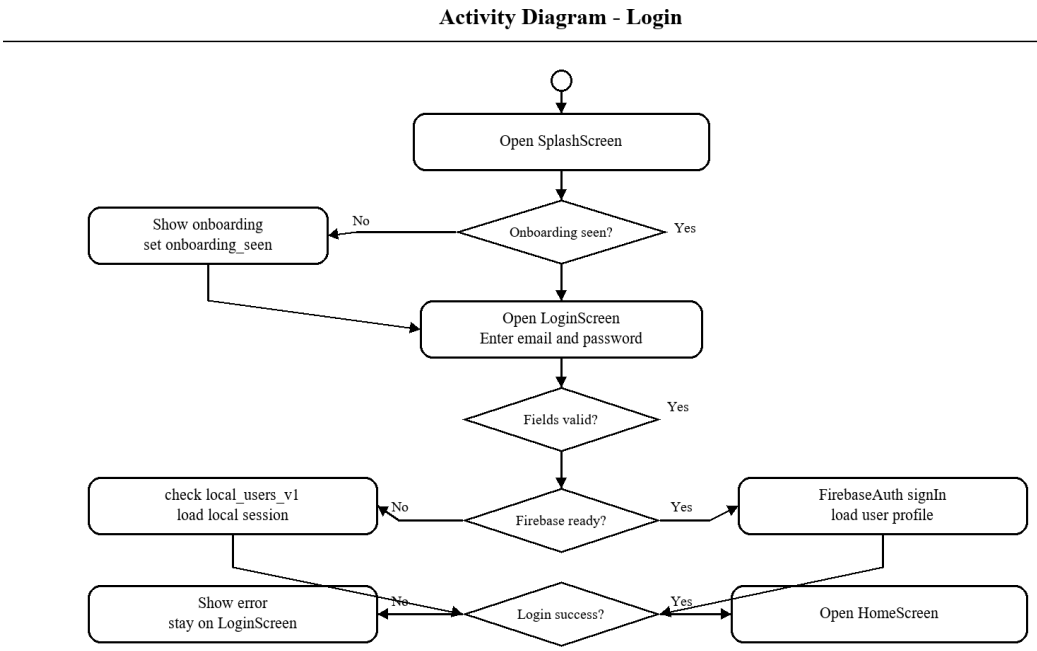


Figure 5 4.5: Activity Diagram for Login

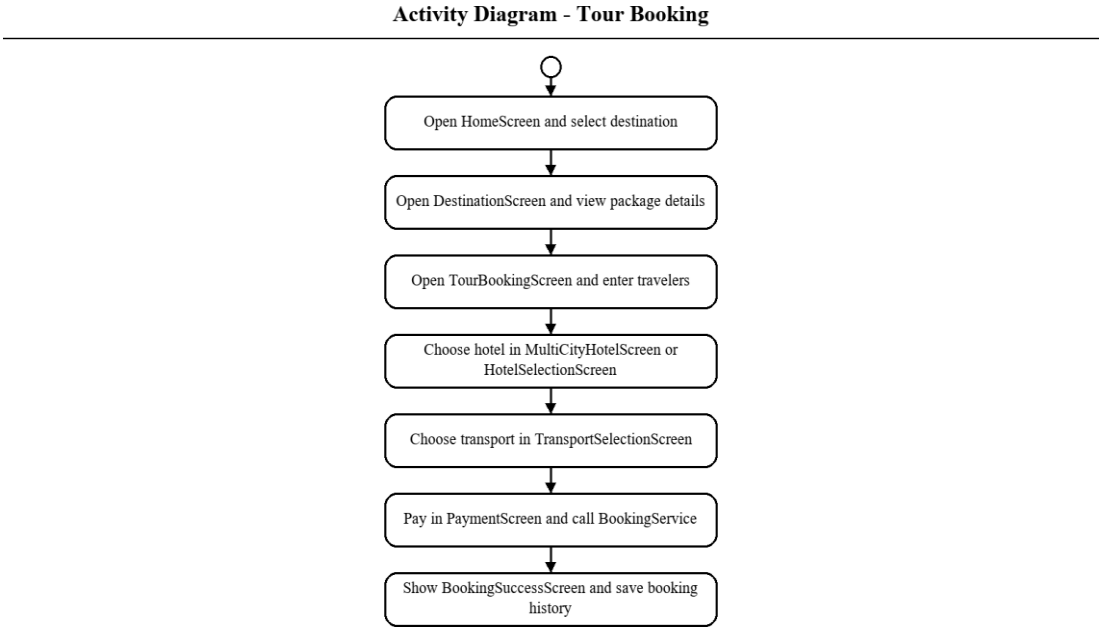


Figure 6 4.6: Activity Diagram for Tour Booking

4.4 Data Design and Firestore Schema

Data design involves both typed models and more dynamic storage techniques. User profiles, destinations, hotels and bookings are kept in collections in Firestore when an internet connection is available. Some typed models such as Destination, Hotel, Tour Package, Booking, Car Booking and Hotel Booking exist to provide structures for the rest of the system and act as the transition from the UI layer and storage layer.

The schema is kept deliberately simplistic for academic manageability whilst still

encompassing the relevant relationship: A user could relate to profile information, bookings relate to certain destination or choices of service, or listings related to the owner ID. Local storage is used in the specific places where the need to manage a set of user-created listing or where a fallback mechanism should exist in the absence of the cloud.

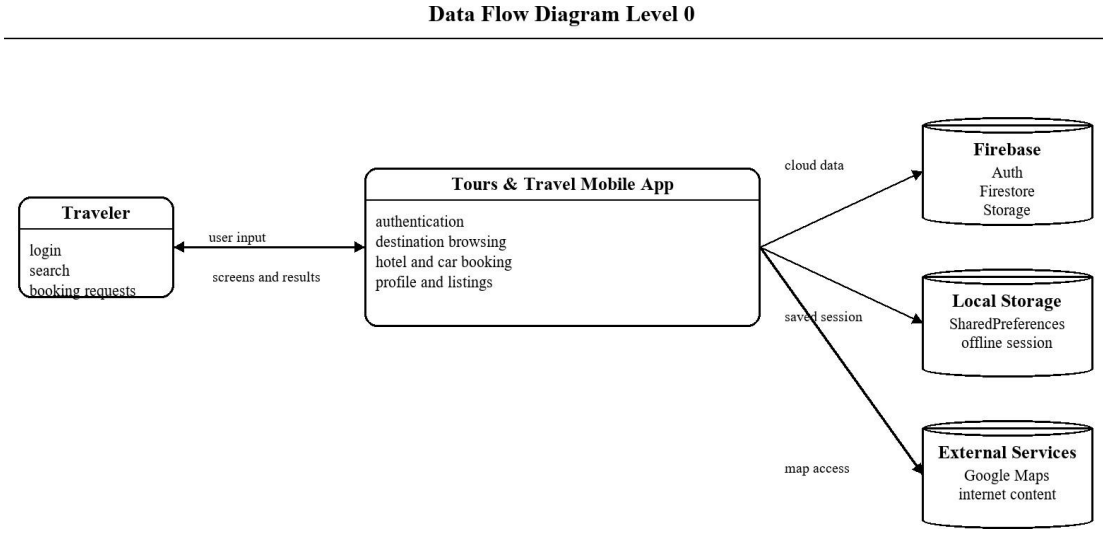


Figure 7 4.3: Data Flow Diagram Level 0

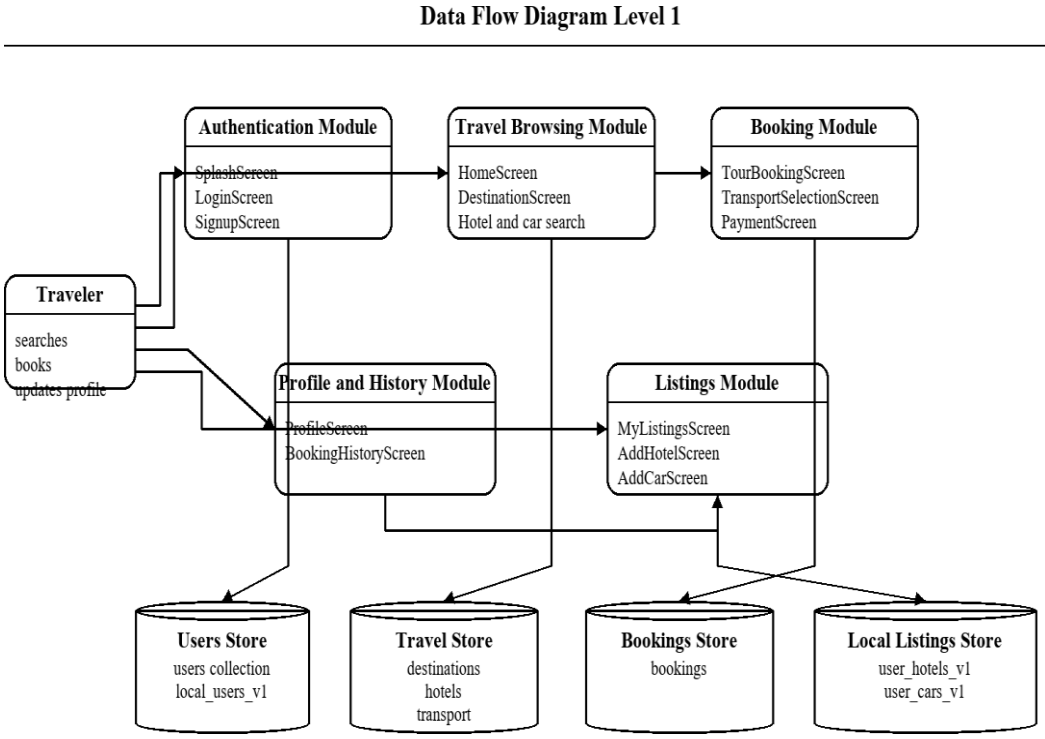


Figure 8 4.4: Data Flow Diagram Level 1

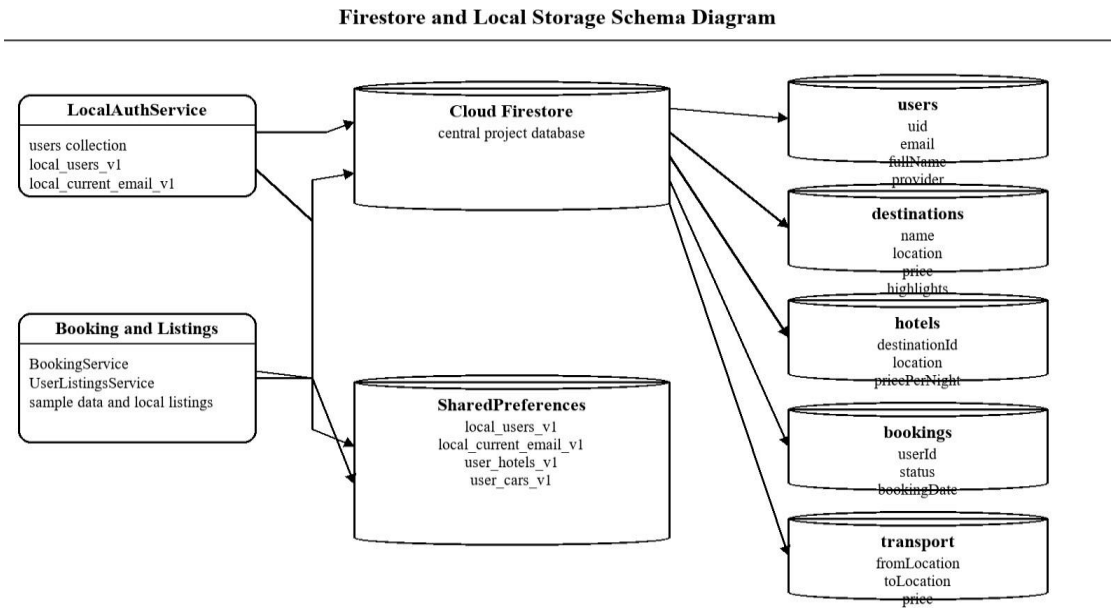


Figure 9 4.5: Firestore Schema Diagram

Table 8 4.2: Key Firestore Collections and Their Roles

Collection	Purpose	Typical Fields
users	User profile storage	uid, email, full Name, created At, profile Image Url
destinations	Tour destination data	name, description, image Url, rating, price
hotels	Hotel information	name, city, price Per Night, amenities
bookings	Tour-related booking records	User Id, destination Name, booking Date, total Price
profile images	Firebase Storage path for user images	file metadata and download URLs

4.5 Class and Service Design

The use of service classes as the coordinators of non-trivial business operations. Local Auth Service, deals with the composite hybrid authentication and login state of the application. Data base Service deals with the acquisition of the destinations, hotels and bookings data. Booking Service deals with operations on bookings. User Listings Service deals with saving hotels and cars locally. Local Booking Store enables a session-level instant view of the confirmed bookings.

This service-based approach towards designing classes gives the codebase a tangible form that students and reviewers can easily relate to. The clear separation between data representations and behaviors helps users understand that models simply contain data (a booking, a hotel) while

services handle behavior associated with them (retrieval, modification, display etc.).

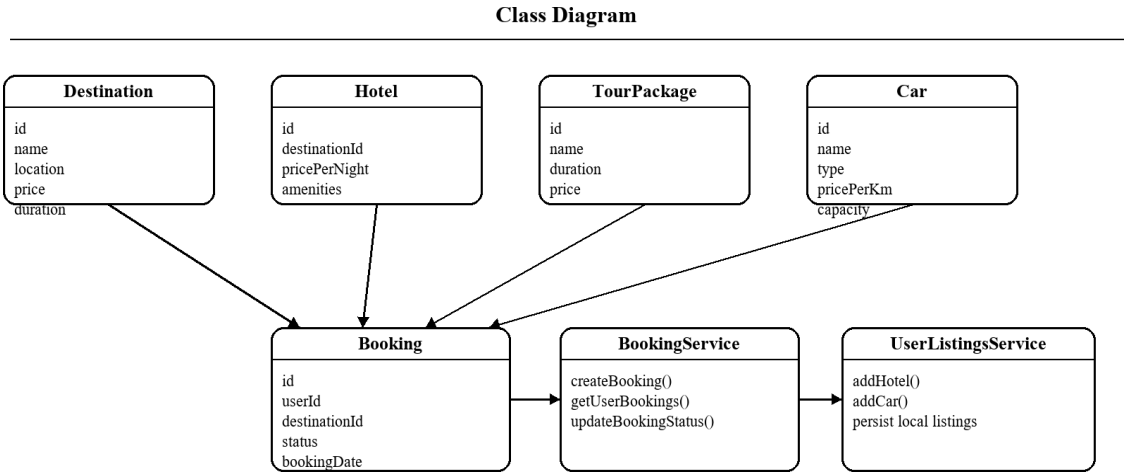


Figure 10 4.6: Class Diagram

Table 9 4.3: Screen-to-Service Mapping

Screen	Primary Services
Login Screen / Sign up Screen	Local AUTH Service, AUTH Service
Home Screen	Database Service, AUTH Service
Hotel Search Screen / Hotel Selection Screen	Database Service
Tour Booking Screen / Payment Screen	Booking Service, Local Booking Store
Profile Screen	Auth Service, Local Auth Service, Firebase Storage
My Listings Screen / Add Screens	User Listings Service
Booking History Screen	Local Booking Store, Booking Service

4.6 Interface and Experience Design

The interface design focuses on cards, gradients, icons, images, tabs and summary boxes in order to establish an elegant travel oriented identity. Appropriate typography, spacing and contrast are implemented for destination, price, ratings and confirmation actions to be effortlessly perceived. Animations add to the sophisticated appearance but remain as a non-critical component for core functionality.

Because destinations themselves are aspirational goods, emotional design plays a vital role in applications oriented towards traveling. Hence the UI is not overlooked but image and motion are strategically used to set the tone and deliver ambiance, all the while maintaining clear interaction rules. This can be observed in the homepage, the destination cards as well as the booking confirmation process.

4.7 Security and Validation Design

Security in the current project predominantly concentrates on providing authentication-secured access, regulated profile operation, and validated form entries. The security boundary is

predominantly secured by using Firebase Authentication for the implemented cloud-based login patterns, whereas validators can guarantee that fields such as email address, password, phone number, and payment type entries correspond to the appropriate pattern.

Even if this is not a product release the design of validation logic is of primary concern as it defines user-trust and prevents ill-fated state changes. The following components form part of the present design: client-side validation, awareness of the local session, and a restricted mode of storage-operations. It will be possible to extend this design to a secure format by adding authorization, encryption, and server-side checks.

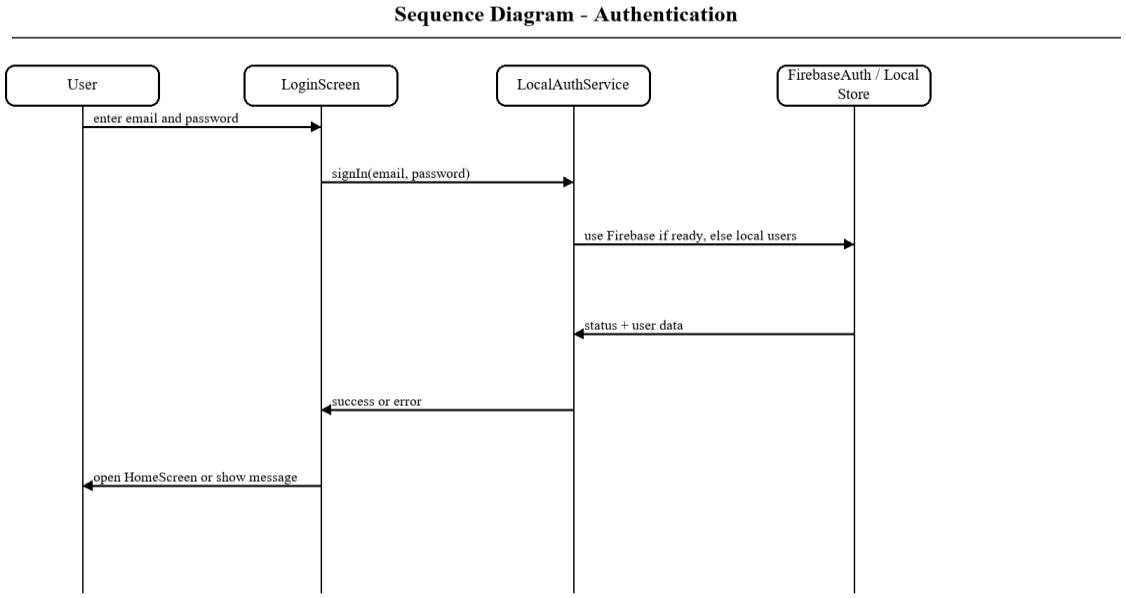


Figure 11 4.7: Sequence Diagram for Authentication

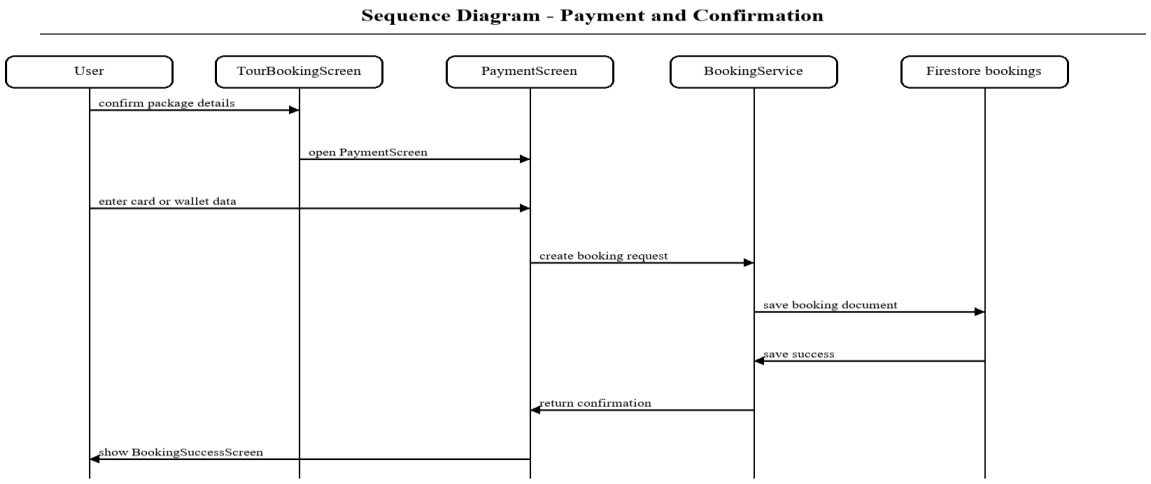


Figure 12 4.8: Sequence Diagram for Payment and Confirmation

4.8 Summary

This stage builds on the requirements and maps them to an actual system architecture. It dictates user interaction within the application, data location, service interaction and diagrams can be used to define the software architecture. Decisions made in this stage influence the next development chapter.

Chapter 5
5 System Implementation

5.1 Development Environment

The application was developed using the mobile framework Flutter and its underlying language Dart. The development was carried out on a standard mobile environment, on either Android Studio or Visual Studio code. Firebase was setup to handle authentication, the Firestore database and file storage. Further packages included, map support, location access, shared preferences, fonts and image display as well as other various functions based around a travelling interface.

The development environment was chosen purely from a software engineering standpoint in relation to productivity and functionality. Flutter enables fast iteration with its hot reload feature, and its' package management system found on pub. dev makes it feasible to include unique functionalities such as integration with Google Maps, picking images, and cached rendering of media.

5.2 Project Structure

The project repository follows the typical structure and is composed of screens, services, models, widgets, components and utils folders. This allows for an easier explanation of the project's features during review due to clear separation of what's user and code facing.

This structure is useful for maintainability as well, since a new developer can easily see how the booking logic is implemented, where the models reside or where the re usable widgets are defined. This will also be very beneficial in a student project where understanding code and being able to explain it is equally important.

Table 10 5.1: Project Directory Summary

Directory	Role
lib/screens	All major user-facing screens
lib/services	Business logic, storage, and backend interactions
lib/models	Data entities and conversion logic
lib/widgets	Reusable generic UI widgets
lib/components	Travel-specific reusable cards
lib/utils	Theme, constants, validators, animations, extensions
assets/images	Destination and visual assets
docs	Project documentation and thesis artifacts

5.3 Authentication Module

The Authentication Module is responsible for delivering a secure signup, signin, signout, and hybrid session persistence. This part of the application helps link domain intent to a targeted sequence of interaction and is therefore critical to how users experience the product's overall value. This module is unlike a simple demo screen, but rather a closed flow that is anchored by a user's intent and culminates with a perceptible state change or feedback message.

The Login Screen and Signup Screen are the main components of this module. A user who opens the application first logs in with a provided email and password or chooses to sign in using social login and then is directed to the home screen following authentication. Users can register their accounts from the signup screen by providing required information such as name, email and password. This step-by-step user flow design is because a user will first survey choices, then evaluate, then book. Summaries, labels, and actionable buttons are all used to assist the user.

On the technical side of this module, we have Local Auth Service and Auth Service. The data and states involved in the module will be managed through Firebase Authentication, a Firestore users collection, and then Shared Preferences in case of failure of Local Auth Service and Auth Service and finally the whole process will be separated from screen rendering. Also it's easy for maintaining; a modification of validation, storage, or server side in the future can be made from the service instead of from the ui code.

The Authentication Module is designed to be both cloud-backed and a fallback. This allows the app to still be shown in an environment where Firebase is partially configured or has partially failed to initialize. It also adds value to the thesis by illustrating how domain-specific decisions in travel software are manifested in real mobile user behavior. The module is therefore both a feature, and a demonstration of software engineering concepts (separation of concerns, data management and flow, user-centered design) in the real world.

Another insight learned from the Authentication Module is the necessity for UI and system behavior to scale together. In other words, neither booking nor a profile related feature are complete with only displayable fields; they must also validate user input, maintain important context, and provide concrete feedback after a given interaction. This project continually demonstrated that a professional app isn't the result of only surface-level stylization but rather the interplay of data logic, speed of response, visual hierarchy, and effective messaging.

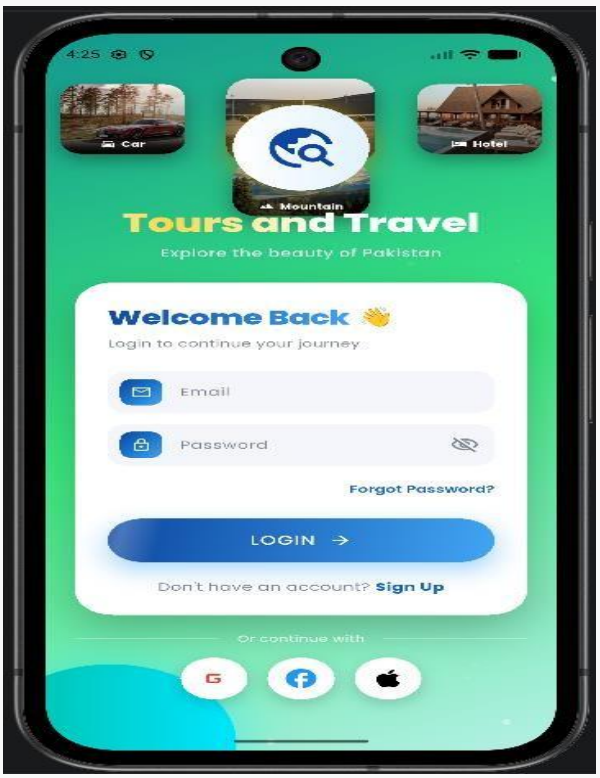


Figure 13 5.1: Login Screen

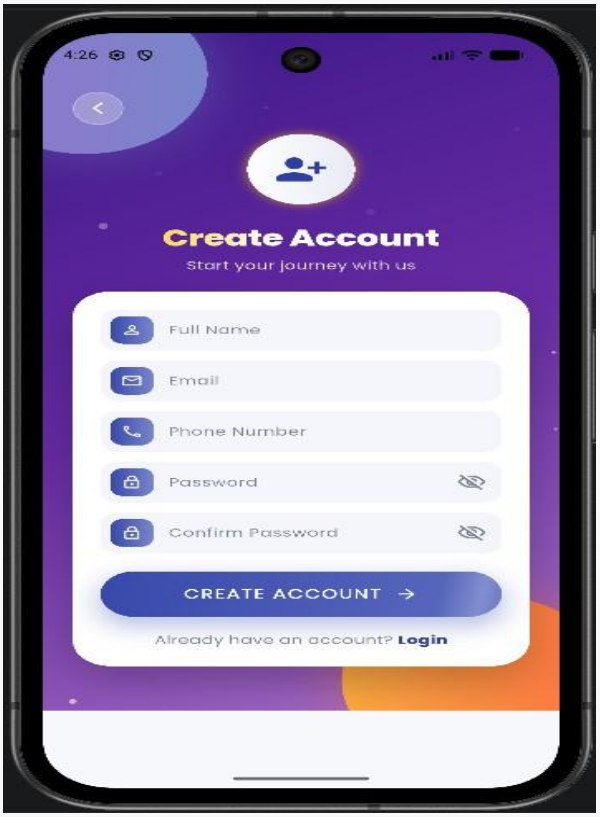


Figure 14 5.2: Sign Up Screen

5.4 Home Screen and Travel Discovery Module

The purpose of the Home Screen and Travel Discovery Module is to display destinations, tour categories, services, quick actions and visual entrances to the remainder of the system. This portion of the application connects the domain intent with a focused interaction sequence and thus plays an essential role in how the user feels about the application's usefulness. Unlike standalone demonstration screens, the module is framed as a complete flow beginning with the user's need and ending with an observable change or informational output.

The primarily screens involved in this module are Home Screen. Upon being authenticated, the user reaches the home screen where single destination trips, multi-city packages, services, and promotional areas are displayed within a rich and visual interface. From this point forward, the user is able to enter the tours, hotels, maps, cars, listings, and profile areas of the application. The design behind this flow is that users typically progress through tasks in a step-by-step fashion-first exploring options, then comparing them and finally making a reservation. Because of this, the module utilizes summaries, labels, and clear call-to-action buttons to guide the user toward success.

Technically, the module utilizes the Data base Service and the Auth Service. Data and state are both managed through Firestore-backed or demo destination data with screen state occurring locally within the screen. This design choice maintains separation between screen rendering and service level logic, making the code easier to reason about and easier to maintain. Future changes related to validation, storage, or server side integration are easily made within the boundaries of the services, rather than being dispersed across UI code.

The home screen provides a dashboard, increasing utility by exposing multiple travel decisions within one space while simultaneously maintaining clear grouping of content and the context of bottom-navigation. In addition, the module aids in proving the thesis in that it directly demonstrates the practical translation of software design choices in the domain of travel into application behavior. It serves as an example of the practical application of concepts such as separation of concerns, data flow management and user centered design.

A further realization derived from the module is that quality of interface and behavior must evolve in tandem. That is, a booking or profile related feature is not finished simply because a field is visible to the user. It must also properly validate user input, maintain appropriate context and provide adequate feedback following an action. This project consistently reinforced the idea that a professional looking application relies on the coordination of data logic, response times, visual hierarchy and appropriate feedback, rather than visual aesthetic alone.

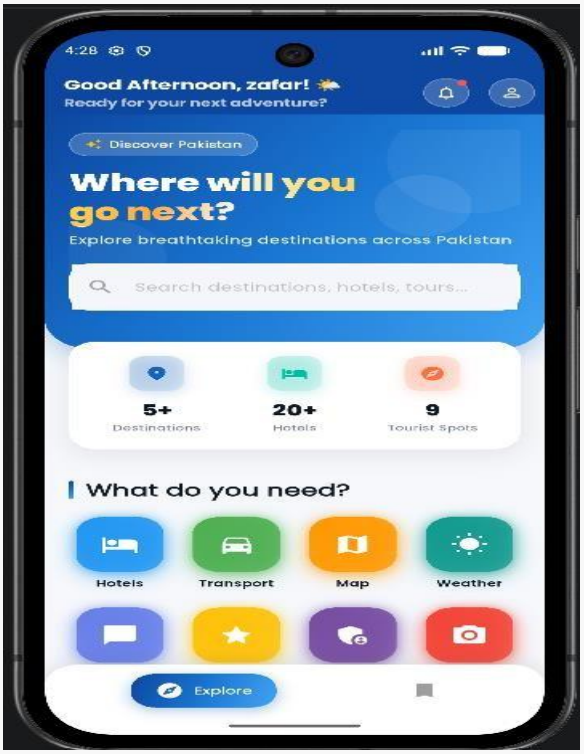


Figure 15 5.3: Home Screen

Figure 16 5.4: Single Destination Cards on Home

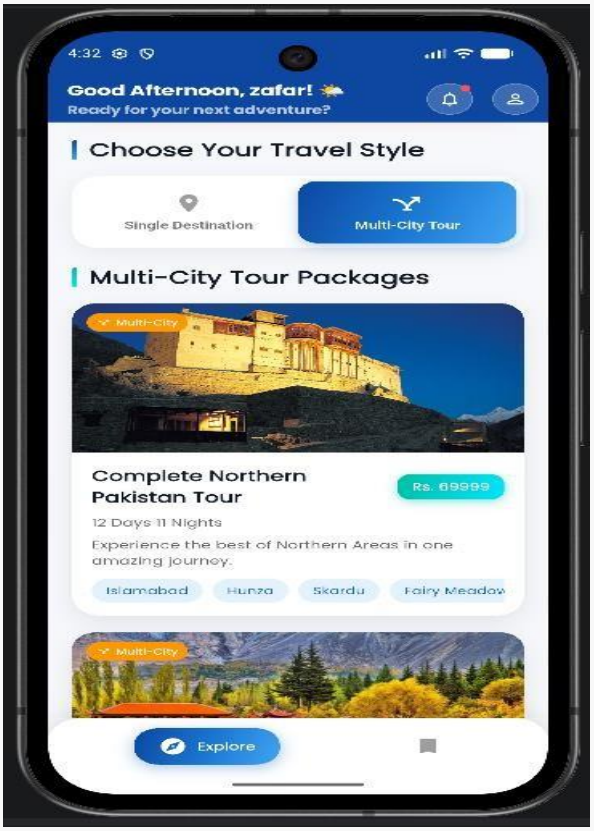


Figure 17 5.5: Multi-City Tour Section

Figure 185.6: Bottom Navigation on Home

5.5 Destination and Tour Booking Module

The objective of the Destination and Tour Booking module is to permit travelers to check destination information and to perform multi-stage tour related bookings. This portion of the application serves to connect the domain intent with a highly focused sequence of interaction and hence plays a crucial part in a user's perception of how valuable the product actually is. Rather than isolated demonstration screens, this module has been conceived as a self-contained sequence that has both a beginning state-a user's need for functionality, and an ending state-a visible state change, or meaningful feedback.

The main screens used for this module include Destination Screen, Multi City Hotel Screen, Tour Booking Screen, and Transport Selection Screen. The user can select a destination or tour package, view a highlights and trip information about the selection, select hotels and transportation options related to it, view a summary of the booking and proceed toward payment. The Multi-city flows assist the user with successive selection of hotels for a multitude of destinations. This sequence of presentation is of significance because a user's interaction is generally sequential; they initially browse options, then compare the possibilities, and finally make a purchase decision. Hence, summaries, labels and clear action buttons have been employed to assist the user with remaining oriented.

In the technical sphere, this module involves Database Service, and Booking Service. Data and state are managed through Destination objects, the data relating to chosen hotel options, booking summary data, and optionally Firestore storage. This allows the screen presentation layer to remain separate from services which deal with the logic of how data is retrieved or manipulated, resulting in a more maintainable code base, since updates such as modifications to validation rules or to server side integrations need not be propagated through all the UI screens.

This module forms a crucial part of the domain identity of the project because it combines both tourism discovery and transactional intent. The presentation relies heavily upon contextual summaries that ensure the user is provided with adequate information and hence does not become disoriented prior to finalizing the transaction. Furthermore, the module contributes to this thesis because it exemplifies how certain software engineering principles used in the design of travel oriented applications can be enacted within a mobile context. It is not only a core piece of functionality, but a manifestation of the way in which software engineering principles, such as separation of concerns, data flow management, and user centered design are manifested in a practical project environment.

One of the additional lessons learned from this module, is the need for interface design and system behavior to progress hand in hand. In other words, just because fields on a screen may be populated with data does not mean that a booking or profile-related functionality is complete; that it must perform input validation, retain useful context, and provide positive confirmation. This project has continually emphasized that what provides an application with a "professional" feel and high user satisfaction is a combination of data logic, responsiveness, information architecture and response messages; rather than solely visually appealing styling.

Figure 19 5.7: Destination Detail Screen

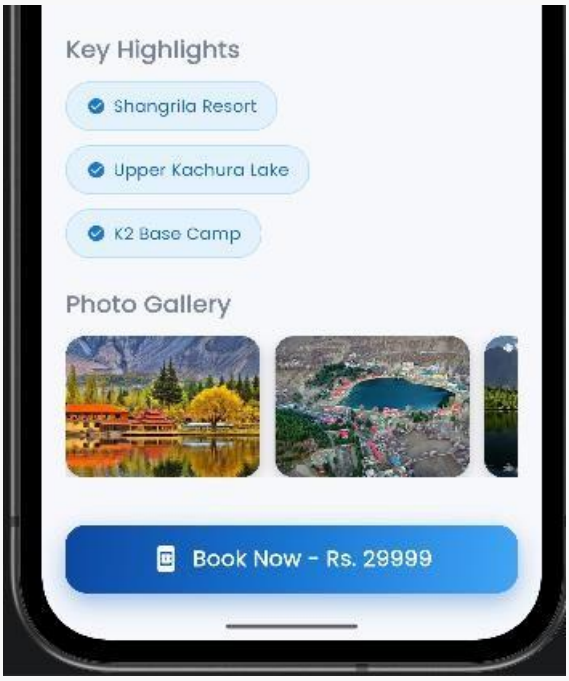


Figure 20 5.8: Destination Highlights and Booking Call-to-Action

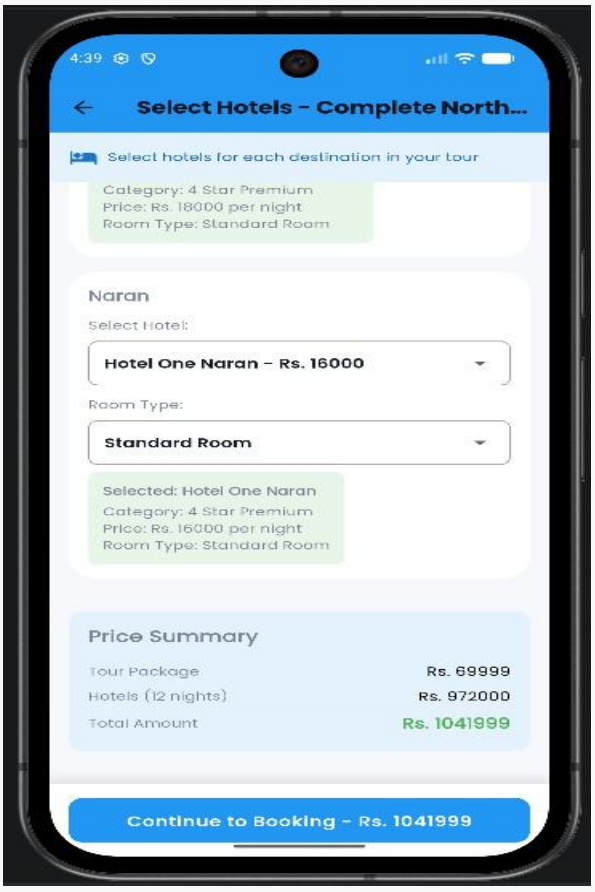


Figure 21 5.9: Multi-City Hotel Selection Screen

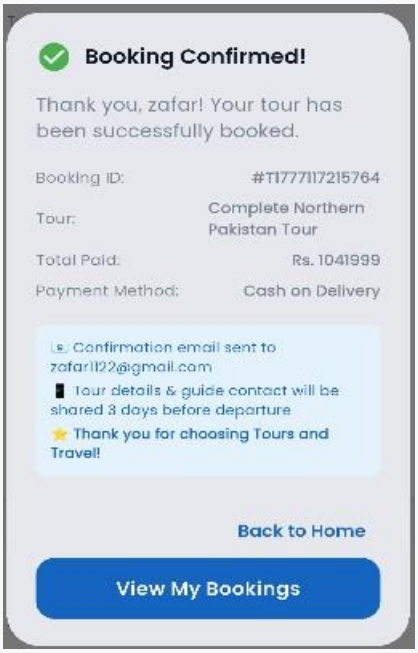


Figure 22 5.10: Tour Booking Summary Screen

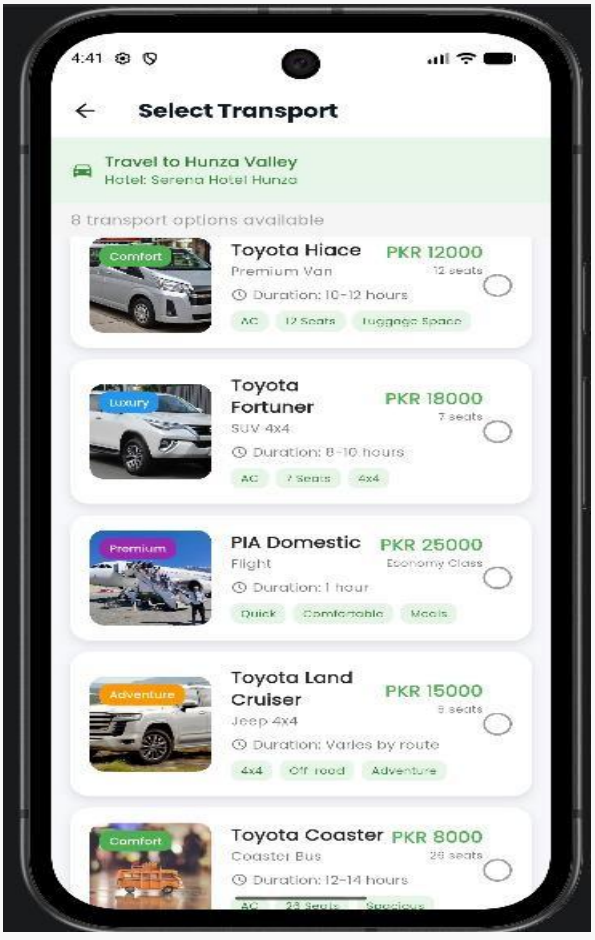


Figure 23 5.11: Transport Selection Screen

5.6 Hotel Booking Module

The goal of the Hotel Booking Module is to facilitate hotel search by city, comparison of hotels, details of the stay, and prepare for the booking. This feature acts as an anchor for mapping domain intent to a single interaction flow, playing a critical role in defining how useful the system is in the eyes of the user. Rather than a series of disconnected demo screens, this feature is designed to function as a complete flow, starting with a need and culminating in a noticeable change or informative response on the UI.

The primary screens within this module are the Hotel Search Screen and Hotel Selection Screen. From the Hotel Search Screen, the user selects the destination or city, sets desired date or guest parameters, proceeds to the Hotel Selection Screen and reviews the hotel results, and begins to approach the checkout before ultimately payment is processed. Both the general "browsing" type flow of hotels as well as a "guided tour + selected hotel" style flow are supported. This separation in flow is considered critical for the system design due to the step-by-step nature of the travel planning process; users will generally look through options, make selections, then consider the booking. Thus the system uses summaries, labels and button indicators to keep the user on track through the Hotel Search Screen and Hotel Selection Screen. From a technical standpoint, this module relies heavily on the Data base Service; the data itself, and the state associated with the system, are handled as Hotel data via Firebase or as dummy records + transient selection state, respectively. Separating data management from the UI is the primary reason this design was chosen for ease of reasoning, and for ease of modification; changing how data is stored, validated, or accessed remotely will only affect the Data base Service layer, and not scattered through the entire UI code.

Hotel booking has been considered a discrete, core sub-domain of the overall application, contributing significantly to the sense of product usefulness due to its frequent use in travel planning. It also demonstrates within this project how the core principles of design can be applied within software for practical behavior. It's not simply a feature but an application of software engineering concepts to a real-world case study.

An additional design takeaway is that the visual fidelity and behavior of the system must evolve together. That is, features of this nature such as hotel booking or profile interaction cannot be considered done merely when they are visualized on screen, but must validate input, maintain context, and respond in some useful fashion after a certain event is fired. Repeatedly throughout the project, a system has begun to appear professionally designed once meaningful feedback, responsiveness, and information hierarchy and data flow logic are developed in concert with more surface-level graphical design.

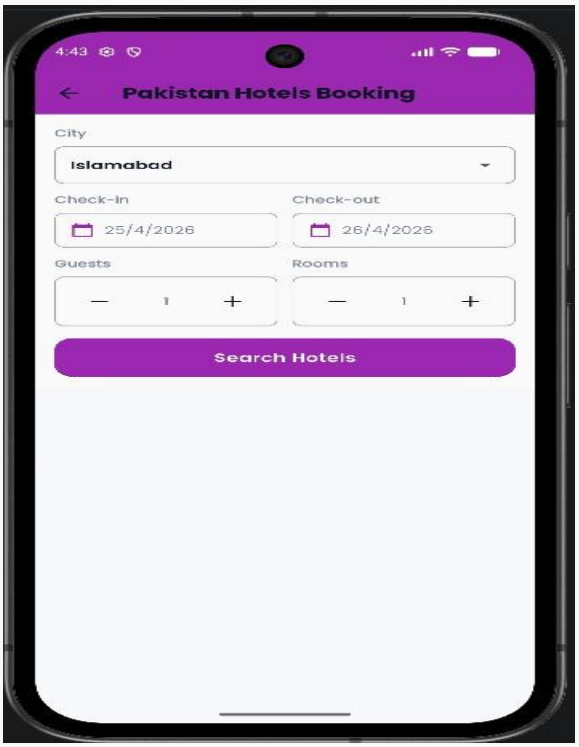


Figure 24 5.12: Hotel Search Screen

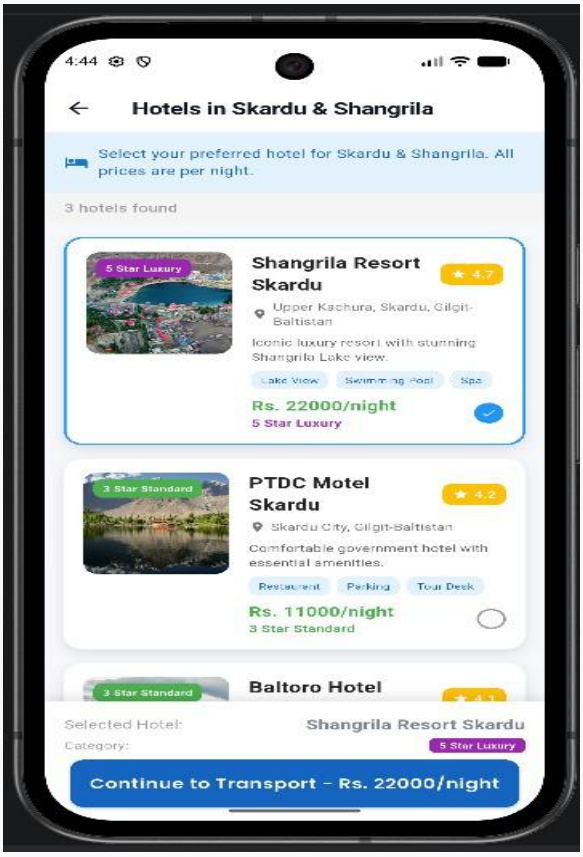


Figure 25 5.13: Hotel Selection Screen

5.7 Car Rental Module

The main function of the Car Rental Module is to implement the functionality to book a vehicle from one city to another, including the fare calculation based on a specified route. It is also the component in the application that directly connects the domain intent to a targeted sequence of interactions, thereby playing an important role in user perceptions about the product's utility. Rather than just presenting separate demonstration screens, it frames the interaction as a complete flow starting from a user desire to the observable change of state, or a meaningful output.

The key screens within this module include the Car Booking Screen. The user selects departure and arrival cities, a desired vehicle, the system allows users to review the prices and any travel related information, before proceeding to the payment section. The system aims to keep the user journey simple, and so make transportation booking available even for users that have not experienced the system before. The interaction design, therefore, uses summaries, labels and prominent action buttons that guide users through the flow.

Technically, the Car Rental module uses Local Booking Store. Data and state are managed through the in-memory booking confirmation state and the local screen state. Keeping the screen renderings separate from the service-level issues helps in having a reasoning system that has good maintainability for the data and state; for instance future changes involving storage, validation and server-side integration would be within the service boundaries rather than sprinkled across the UI code.

The module helps extending the application from the discovery and destination selection aspect into other travel related functionalities. It also showcases how a separate service categories may be incorporated without needing a second application environment within the platform through modular screen design. Overall the module proves the application of certain design principles for travel software applications into observable mobile application behavior and provides contribution to the thesis that software design decisions in the area of travel may be transformed into application functionalities. It represents not only the functionality it implements but the process of translating software engineering design principles such as separation of concerns, state management and user-centered design to practical application behavior in a real product.

From a practical standpoint, the module illustrated another important realization throughout the development of the project – that the presentation layer must closely reflect system behaviors. Essentially, the application does not consider an action complete, even with relation to profile or booking functionality, just because fields are shown on the screen: they must also

validate inputs from users, maintain relevant context, and produce the correct outcomes following the actions from the users. It was a theme recurring throughout the project development that what made an app feel polished is not its visual style alone, but a correct marriage of data logic, response timeliness, visual hierarchy and user feedback.

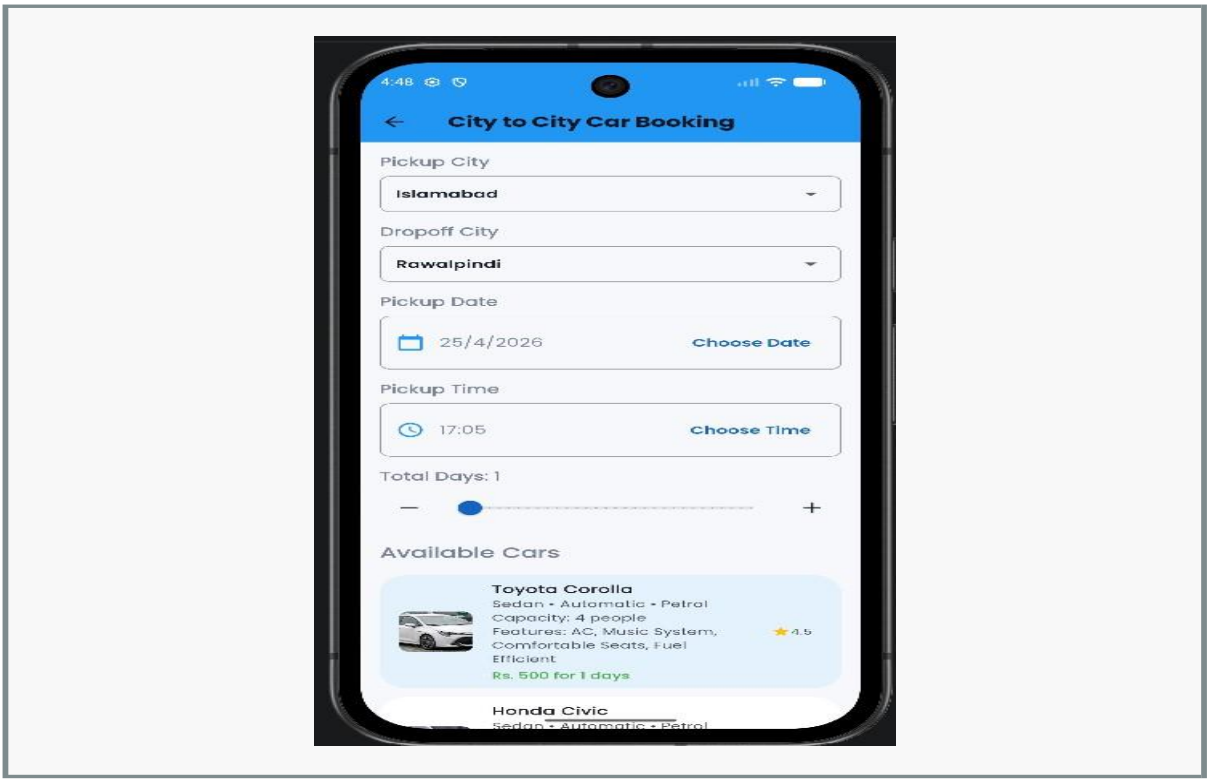


Figure 26 5.14: Car Booking Screen

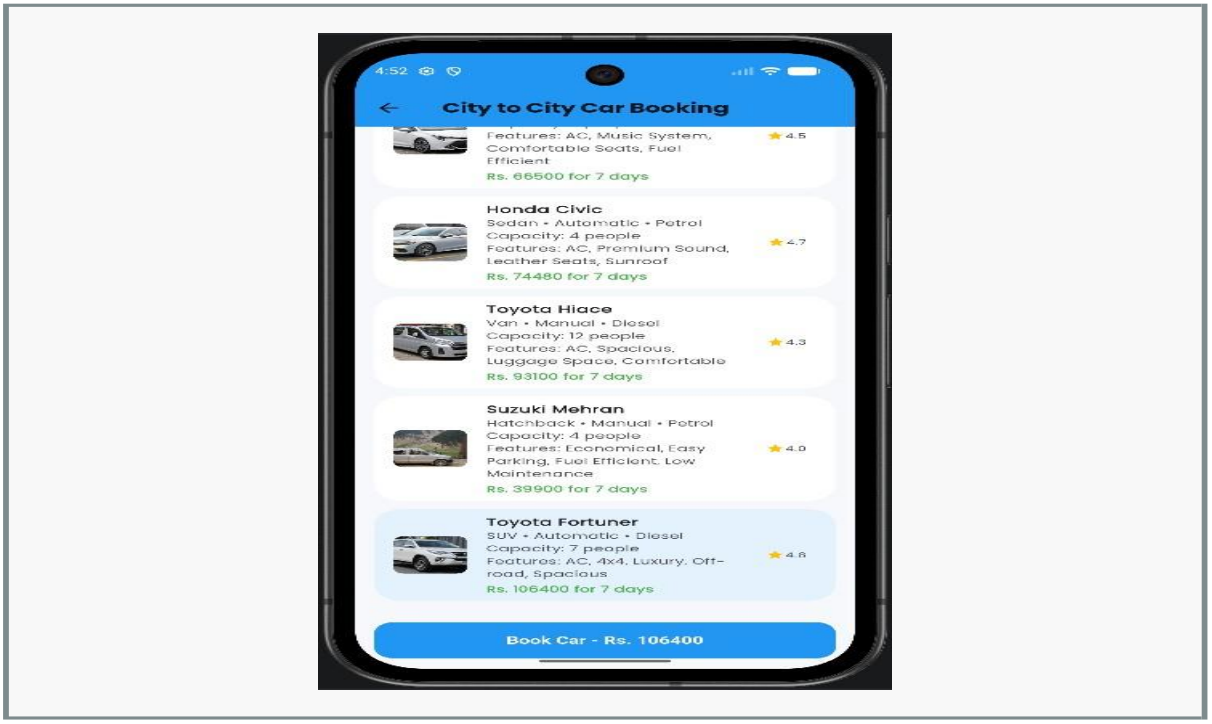


Figure 27 5.15: Car Fare Details

5.8 Payment and Booking Confirmation Module

The module which simulates transaction fulfillment and provides instant user feedback on booking success. It establishes a link between domain intent and focused user interaction flow and therefore represents an important aspect for how users perceive product usefulness. Instead of disconnected presentation screens it is defined as self-contained flow with a user goal and a tangible outcome.

Key screens in this module are Payment Screen, Booking Success Screen and Booking Confirmation Screen. The payment screens are reached after service selection and represent the confirmation page showing all booking details, as well as the choice of the payment method. Upon form submission the booking and payment is simulated by the app, confirmed and stored in a local booking store which in turn makes the booking visible on the booking history screen. This step-wise user flow design is of significant importance, as users generally have the mindset to perform actions step-by-step; to browse, to compare and then to commit the booking, thus the module uses summaries, labels, and clear action buttons to keep the user aligned with the context.

Technically the module depends on Local Booking Store and Booking Service. Data and state are passed between screens using session booking records and the currently chosen payment method information. It allows decoupling the concerns of screen rendering from service level logic and therefore leads to better readability and higher maintainability, as future changes in validation, storage or external (server-side) integration can be carried out within service scope, as opposed to being scattered in UI code.

Although the project relies on mocked payment processing and not real financial transactions, the interaction flow serves as a realistic example that such payment services can be integrated later without redeveloping the adjacent user flow. More importantly the module contributes to the thesis by demonstrating how software design decisions in the context of a travel application can be translated into actual mobile functionality. It is not just another implemented feature but rather evidence of how software engineering concepts like separation of concerns, data flow and user-centric design are applied in a real software development project.

An additional realization drawn from this module is the need for interdependence between quality of interface and behavior of system. Namely, features such as creating a booking or a profile are only completed if the input is validated, context is maintained, and relevant feedback is given after an action is executed. This application reinforced the fact that quality software is not just about polished surfaces; it is the synergistic interplay of data logic, response times, visual hierarchy and feedback messaging.

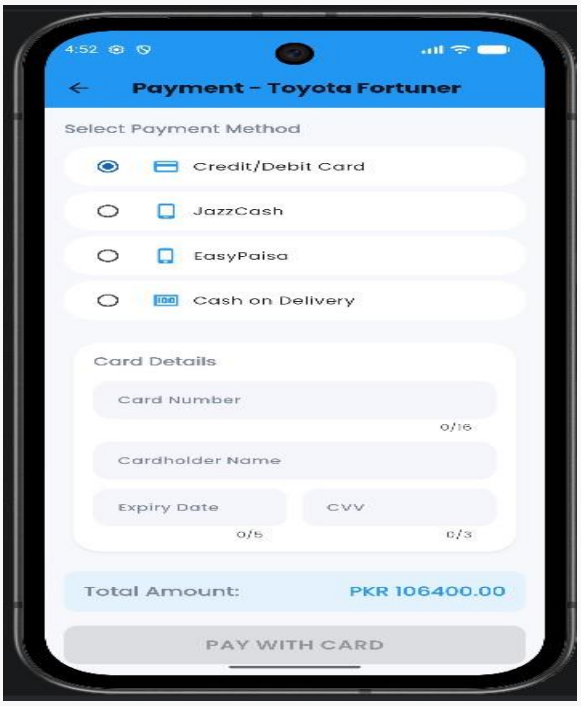


Figure 28 5.16: Payment Screen Using Card Method

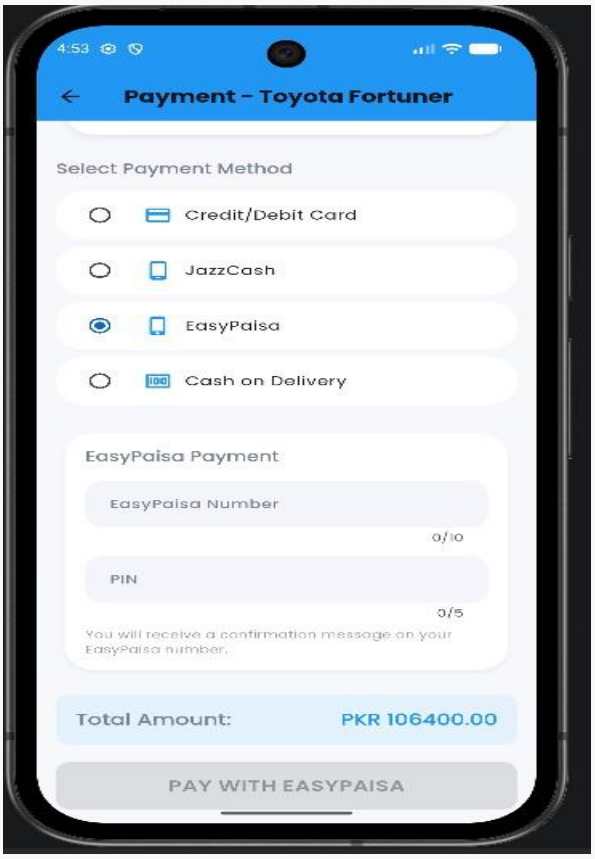


Figure 29 5.17: Payment Screen Using Wallet Method

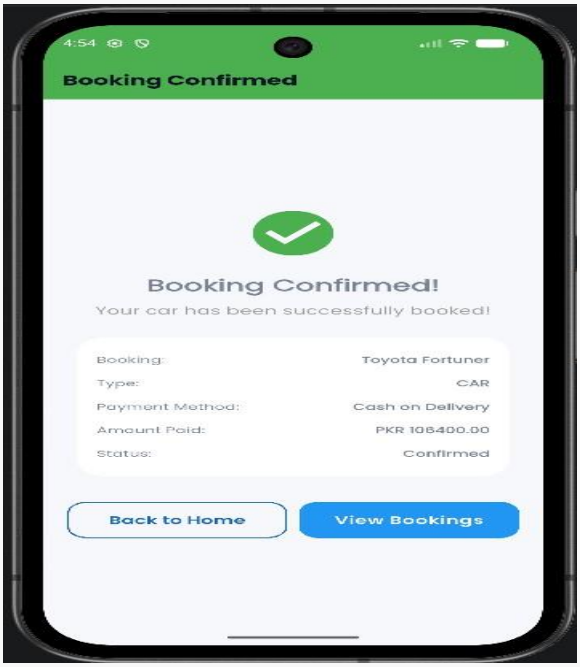


Figure 30 5.18: Booking Success Screen

5.9 Booking History Module

This is a module that presents a tabulated viewing history of confirmed bookings. As it serves to link a domain intent to a sequential, targeted interaction and so plays a role in overall perception of product usability, this aspect of the application is designed not as disconnected demo screens but as self-contained flows beginning with a user need and concluding with a visible state change or piece of information.

The screens involved in this module include: Booking History Screen. Users navigate to this view of confirmed bookings, switch between tabs showing all bookings, tour, car, and hotel booking confirmation history, and briefly examine the details of past reservations. Structured cards containing a booking's basic information are displayed, arranged by tabs, which aid the readability and structure of the records, and a flow design that caters to user habits by beginning with exploration, progressing to comparison, then to booking, by presenting concise summaries, labels and explicit action buttons is used.

Technical details involve Local Booking Store and Booking Service; the storage and data are based on local booking records with session context, and optionally on Firestore booking streams, so that logic is divided between the service and screen layers. The architecture ensures that rendering of the screen remains separate from the logic contained within the service and is therefore easier to trace and manage, because future changes to input validation, database structure, or server-side integration can be managed from service boundaries without necessitating modifications scattered across the UI.

This module adds continuity to the application and reinforces the notion that the application is not solely designed to acquire input. It displays the outcome of the user's actions, adding a more finished, product-level dimension to the system. Furthermore, the Booking History Module contributes to the thesis by offering an example of how the decisions made in designing a travel-specific application can be manifested in real-time behavior on a mobile device, serving as a functional element in its own right and also demonstrating principles of software engineering such as separation of concerns, state and data management, and user-centered design in an active project.

A design takeaway from this module is that user interface quality should coincide with system behavior. That is, features such as creating a booking, or editing a profile are not considered complete when only elements for user input are presented on a screen: data validation, preserved state, and constructive user feedback must follow any performed action. Repeatedly through this project, we observed that professional-level applications can only achieve this impression by combining data logic, timely responsiveness, appropriate visual hierarchy and helpful feedback, not just shallow UI styles.

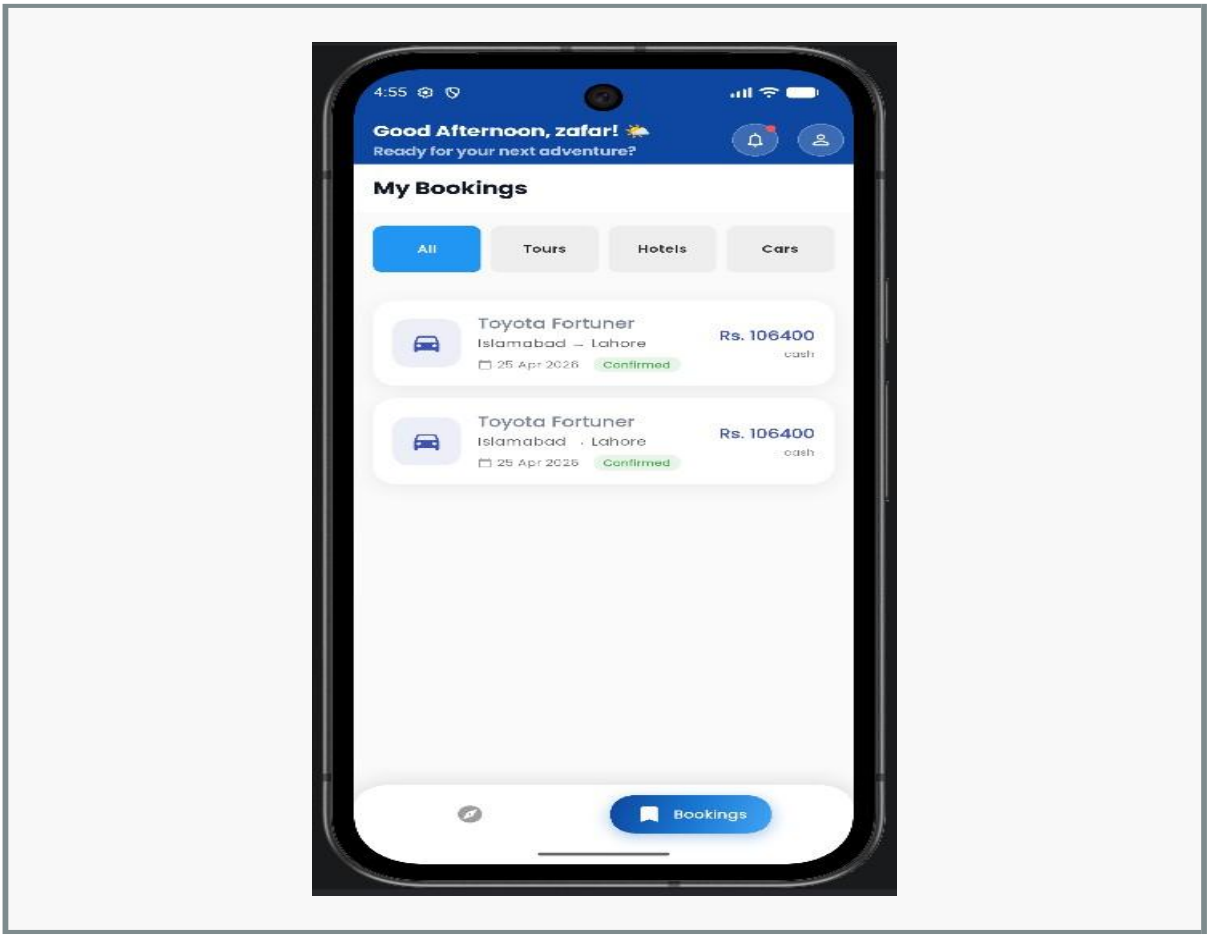


Figure 31 5.19: Booking History Main View

5.10 Profile Management Module

This module is used to manage personal information and set some personal configuration, profile photo upload and so on and signing out the system. This part of application is a very key part of the product and directly influences users to experience product's usability, as it brings domain intent and focused interaction flow. Instead of isolated demo screens, it's designed as a whole system from user's requirement to the visible state changing.

Screens are: Profile Screen. Users can check identity information, change settings, upload a profile picture, sign out from the app. It also includes some user setting regarding language or mode to personalize. The reason it's designed this way is that users normally move by steps, to browse, to compare, to book. Using summaries, labels and clear action buttons to guide user to follow steps are key to this module.

Technically speaking, the module depends on Auth Service, Local Auth Service and Firebase Storage, data and state management use firestore profile updating, firebase storage url for images and shared preferences for user preferences. This separation of screen rendering and service level design is beneficial for the reasoning of code and maintenance, as future updates to validation, storage, or backend interactions can occur within the service without impacting the UI code scattered throughout the UI component tree.

It's important to handling the profile, as this provides the foundation for the rest of the system with a persisting user identity and covers file upload, remote update and local preference management in a single module. The contribution to thesis in this module is showing how decisions on software design regarding travel application can translate into mobile behavior. The module isn't just a functional feature but the concrete illustration of how separation of concerns, data flow control and user-centered design work together.

Additional development consideration from this module is interface quality and system behavior have to evolve together, meaning that a booking or a profile related feature should not just only be composed of some fields displaying on the screen, but also need to perform input validation, maintain context of use and give feedback when user makes an action. In this project, it was learned that professional experience can be gained by a good data logic, responsive timing, visual hierarchy and feedback messaging; but not just by the looks of the application.

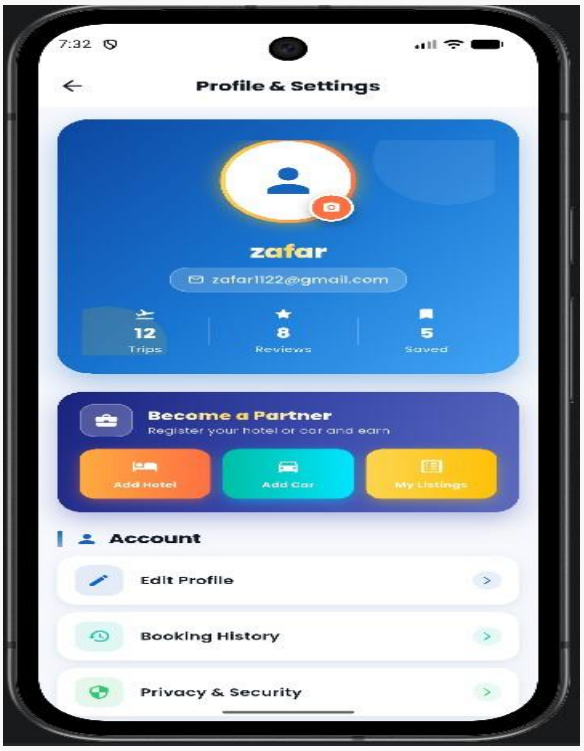


Figure 32 5.20: Profile Screen

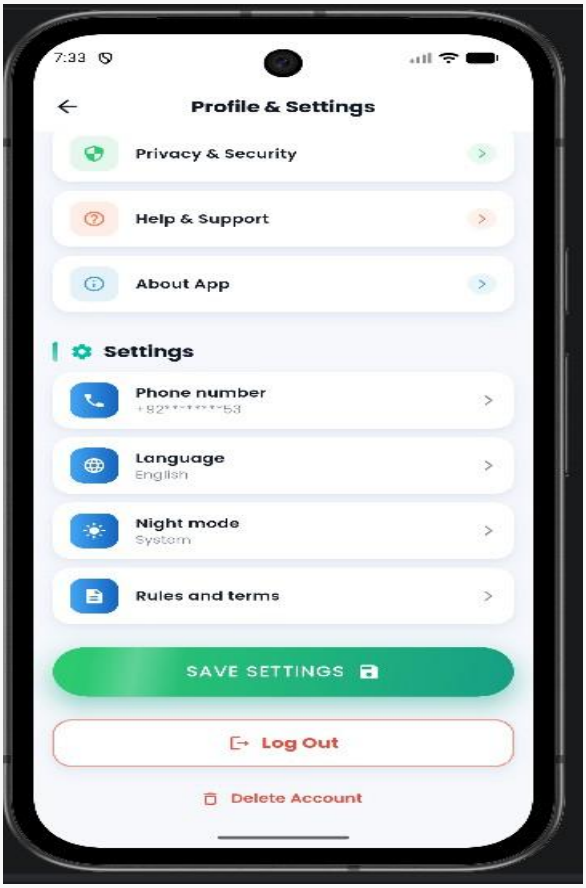


Figure 33 5.21: Profile Image Upload or Settings Area

5.11 Maps and Location Module

The goal of the Maps and Location Module is to enable guided tourism via visualization and placement markers on the map. As it is the interface between the domain intent and a specific interaction flow, the Maps and Location Module is central to the usability perceived by the user. Unlike stand-alone demonstration screens, the Maps and Location Module presents a self contained interaction flow with clear start and end points- from user request to final state change or informative result.

The screens involved in this module primarily are Map Screen. When using this module, a user will navigate to the map screen, request location permission, view their current location (if permitted), browse preset tourist spots such as Hunza, Skardu, Swat, Murree, Lahore, Karachi, and Islamabad, etc., and receive detailed place information by tapping on a specific marker. The flow has been designed to take users through the process step-by-step, and encourage step-by-step thinking since users will most likely look for travel options, compare travel options, and book travel. Clear labels, summaries, and action buttons help the user proceed through these steps.

From a technical aspect, the module depends on Geolocator and Google Maps Flutter. Data and state are handled via runtime location permission state and through pre-programmed tourist spot data stored in code. These decisions have ensured that screen renderings were separated from the actual service-level concerns of data management, which also facilitates code reasoning and improves easy maintenance since further work on validation, storage, or server-side integration would have to be done within the services rather than scattered within the UI code.

By introducing this module, the app extended its services beyond the domain of bookings and added elements of travel assistance and awareness of places. Furthermore, the module enhanced the app’s overall identity as a tourism app and demonstrates integration of the available device functionalities. This module demonstrates to the thesis that design principles of travel applications can be mapped to user actions. It is a demonstration not only of functionality, but of software engineering concepts being implemented on a real-life project, such as the separation of concerns, management of data flows, and user-centered design.

An additional observation from working on this module is the concept of interface and system behavior growing hand in hand. For example, in terms of booking or profile-related features, a displayed form on the screen doesn't suffice for a completed feature. Validation of the user’s inputs and preservation of context is required along with informative results when actions are taken. This project repeatedly provided insights that it takes more than simply adding visually

appealing design and UI elements for a system to be professional; a professional-looking app is the integration of data logic, appropriate timings, hierarchy and visuals, and a well designed feedback message to the user.

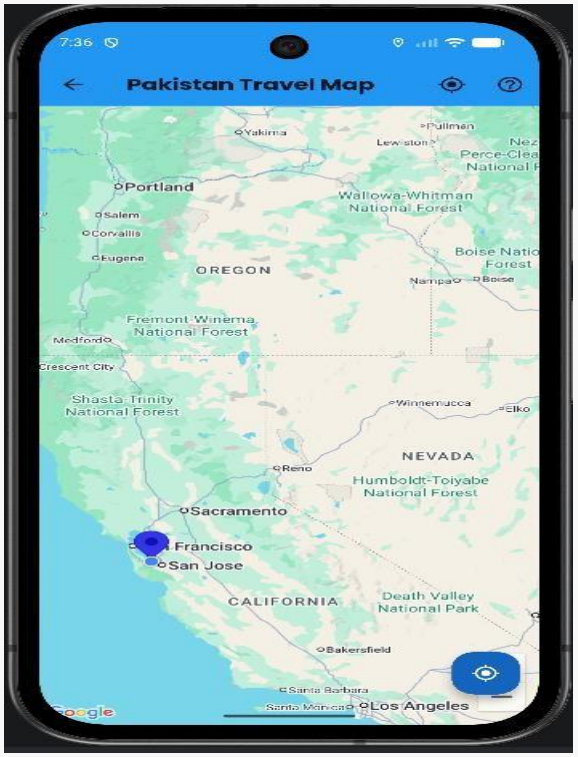


Figure 34 5.22: Map Screen

5.12 User Listings and Marketplace-Oriented Features

The User Listings and Marketplace-Oriented Features allows an authenticated user to list his or her hotels and vehicles in the app. This section of the app brings domain intent and focused flow together and thus is of significant importance to the perception of usefulness of the product. Instead of disjointed screens, the section is modeled as an independent flow with a user goal and an observable state change or informative output at its completion.

The primary screens of the module include Add Hotel Screen, Add Car Screen, My Listings Screen. Once a user logs into the application, the Add Hotel or Add Car screen is displayed to the user who then inputs listing information to the application and later views it within the My Listings screen. This aspect creates a marketplace oriented feature to add to the traveler-oriented aspects of the application, and demonstrates local storage of user created data. Flow design for the user is critical in that the user will tend to go step by step through processes; first by exploring available options, then by comparing options, and finally by purchasing/booking something. Hence, summaries, labels, and informative buttons were used to help the user to navigate through the module.

Technically, the module is made up of User Listings Service, Local Auth Service. All data and state is managed by local storage based on Shared Preferences for all user-generated data. This decouples screens from services making them simpler to reason about and more easily maintained as all validation or changes in storage (or server-side integration in the future) need only be handled in service bounds rather than spread throughout the UI code.

The module is academically valuable in that it switches the application paradigm from solely a consumer oriented app to one with two sides: the consumer and the provider. Although the listings are stored locally rather than online, it demonstrates the service-provider oriented nature of an application. Furthermore, the module demonstrates to thesis readers the applicability of the decision of how travel centered software should operate in a real application context. The application is not only a functional tool, but an illustration of software engineering practices such as separation of concerns, data flow, and user-centricity being applied in practice.

Another learning from this module is how interface quality and system behavior must be implemented simultaneously; a booking or profile related feature is not done when the user sees blank fields, it is finished when the user is validated, her context is saved, and he or she receives useful output. The feature emphasized how to produce a quality feeling app through not just visual design, but also with data processing logic, response time, visual hierarchy, and response messaging.

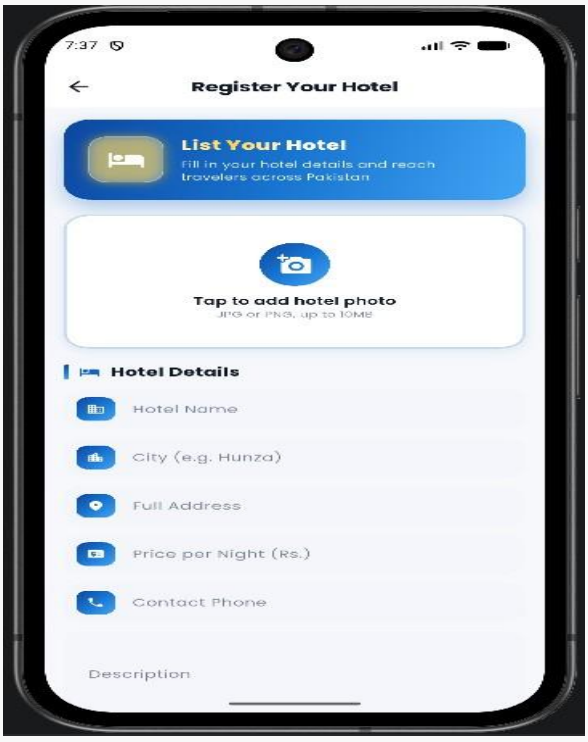


Figure 35 5.23: Add Hotel Screen

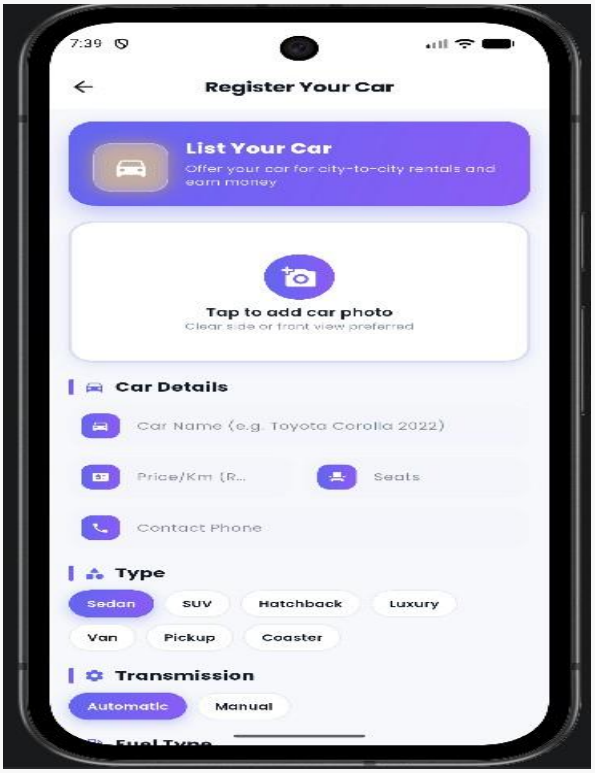


Figure 36 5.24: Add Car Screen

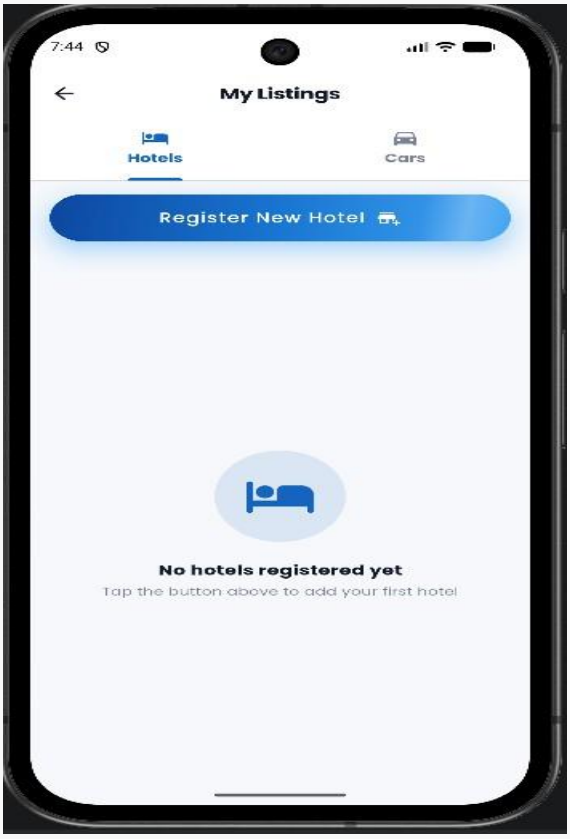


Figure 37 5.25: My Listings Hotels Tab

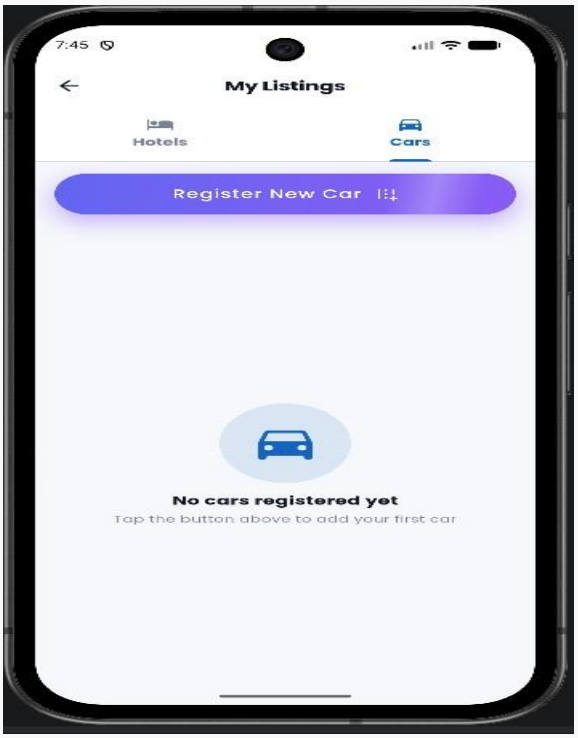


Figure 38 5.26: My Listings Cars Tab

5.13 Supporting Utilities, Animations, and Data Initialization

This supporting utilities, animations and data initialization is intended to increase the visual appeal, the developer support and the readiness of the demonstration. This module of the application is bridging the domain intent and an interaction sequence, and for this reason it plays a crucial role in how useful users feel the application to be. In this context, it's defined as an atomic interaction rather than a group of demo screens. It's seen as a workflow: it starts from a user need and ends with a visual state change or an informative outcome.

Firebase Data Initializer Screen and Photo Gallery Bottom Sheet are the primary screens within this module. Developers or evaluators might initiate the sample data from the first screen while the end-user might see animated transitions, card motion, shimmer loading styles, or a visually appealing gallery section. This workflow definition is very important in terms of user experience, because it mimics the natural workflow an end-user typically follows: exploration, comparison and then booking. Thus, the module aims to lead the user along using summaries, labels and distinct actions.

The module requires two external components from the system: Data base Service and animation. dart utilities, alongside the constants. dart file. Data is primarily dealt with through the two mechanisms: seeding with a Firebase seed data creation and runtime managing UI state with state variables in the dart code. This definition separates screen rendering from service

level implementation and leads to an easier maintenance and readability of the system. Modifications with regard to validation, persistence or server interaction could thus be isolated and localized to only services rather than spread throughout the entire UI code.

These supporting utilities are not less important than the primary system functionalities. In a student project demonstration, the visual quality and smoothness of the UI are highly effective, and the presence of seeding data allows to conduct a complete presentation and to test a well-formed application. Also this module contributes to the thesis to some extent, to illustrate the decisions in designing travel software and in how they map to a real mobile behavior. It is indeed not a mere functional part of the system but also a demonstration of applying computer science concepts, like separation of concerns, state management and user-centered design, in the context of a practical system.

The final takeaway from this module is that interface quality and system behavior are tightly intertwined. A booking or a user profile related function isn't complete just because you can see its fields in the user interface: you must also validate their input, keep the relevant context, provide effective feedback on performed actions and more. This project in general proved to me that app quality isn't defined by the looks only, but it is a result of interaction speed, the logic and the responses combined together with visual cues and informative messages.

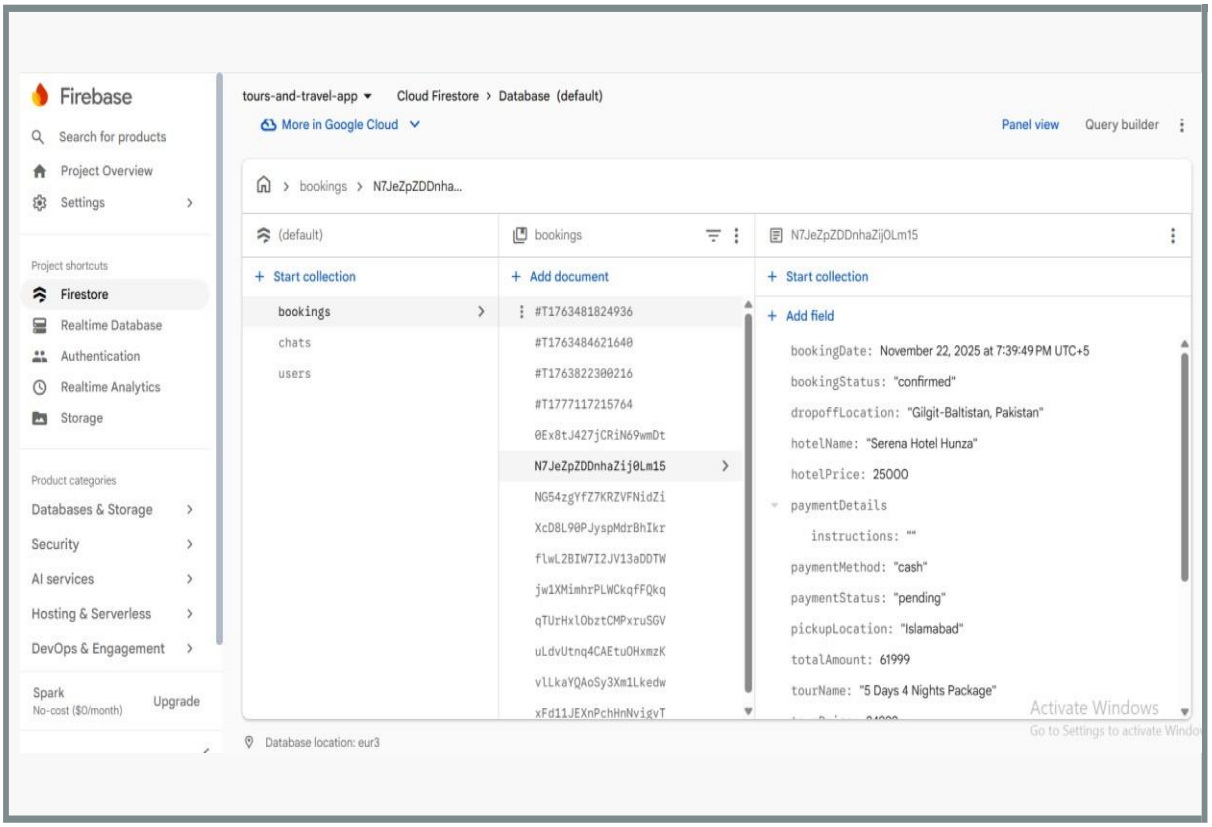


Figure 395.27: Firebase Data Initializer Screen

Chapter 6
6 Testing, Validation, and Evaluation

6.1 Testing Strategy

Testing for this project is a combination of structural thinking and manual validation. Since this application is comprised of several inter-connected UI flows, testing focuses on checking end-to-end paths, validating form inputs, verifying consistent navigation, and testing local storage under various cases. This is a strong testing strategy for this project because a student mobile app must be both functional and easy to use.

The total test strategy spans from authentication, navigation, presentation of data, simulation of payments, local storage, to cloud-integrated functionality. Testing involves positive and negative test cases, specifically involving data input, sessions, list creation, and filtering booking history. The objective of testing in this project is to verify not just that the application handles the expected (happy path) flows, but also obvious invalid cases gracefully.

6.2 Unit-Level Observations

There are a small number of formal unit tests at the unit level in the current repository, which is itself an important finding to note in this thesis. However, because validators, models and service methods are so modular, they lend themselves well to being unit tested; and unit testing in these areas is highly recommended going forward. Specifically, areas such as email validation, card input validation, model conversion and local list serialization would benefit from being unit tested independently.

This observation identifies a frequent distinction within academic prototype projects: they may present believable user flows, yet still have limited automated validation to deem them "production ready." Acknowledging this deficiency is necessary because it displays an honest analysis as opposed to an exaggerated one.

6.3 Integration and Flow Testing

Integration testing verifies whether the services, stored states, and screens interact properly. Login to home, home to booking, payment to history, profile picture update, and storage of add-listing to my listings are all examples of what's covered. These are joins between the modules, not individual tests. Travel system in particular depends highly on continuity to inspire confidence in users, as just one failing transition, or lost summary, will reflect badly on the entire system, and therefore a considerable amount of focus is given to it in the evaluation

narrative in this thesis.

Table 11 6.1: Manual and Integration Test Cases

Test ID	Scenario	Expected Result	Status
TC-01	User opens login screen	Login form renders correctly	Pass
TC-02	User enters invalid email	Validation message appears	Pass
TC-03	User signs up with valid details	Account creation flow completes	Pass
TC-04	User logs in with valid credentials	Home screen opens	Pass
TC-05	User opens destination details	Destination information is shown	Pass
TC-06	User starts hotel search	Hotel records or demo data appear	Pass
TC-07	User selects multi-city package	Multi-city hotel screen opens	Pass
TC-08	User confirms tour booking summary	Payment screen opens	Pass
TC-09	User chooses wallet payment method	Relevant fields are displayed	Pass
TC-10	User confirms payment	Booking success state appears	Pass
TC-11	User opens booking history	Confirmed booking card is visible	Pass
TC-12	User uploads profile image	Image upload flow starts and feedback appears	Pass
TC-13	User opens map screen	Markers and map view load	Pass
TC-14	User adds hotel listing	Listing is saved locally	Pass
TC-15	User adds car listing	Listing is saved locally	Pass
TC-16	User reviews my listings	Saved listings are visible in tabs	Pass
TC-17	Firebase unavailable at startup	Fallback auth flow remains possible	Observed
TC-18	Search bar interaction	Visual field available, feature expansion pending	Partial

6.4 Manual and Usability Testing

Tests were conducted manually to ensure all forms, cards, buttons, images and transitions worked as expected on typical user scenarios. Readability of the price summary, accuracy of labels, click-ability of the buttons, presence of actions such as log-in, book, save and confirm on important sections were evaluated.

Usability-focused testing also considered the ease of comprehension for a first time user: to ascertain whether a user understood the task expected from them upon seeing each of the key screens. This application makes use of cards, icons, groups of elements on one hand, but explicitly defines the action required of the user using buttons.

6.5 Defects and Improvement Findings

Issues also emerged, such as incomplete functionality. One booking sequence uses volatile, in-memory state and doesn't seem to persist to the cloud, and the existing automated test file is still the default one from the Flutter sample instead of a suite specific to this project. Although these issues don't detract from the project's value, they establish the divide between what would be appropriate for a prototype and what would be production ready. Such issues are useful for academic purposes as they signify the approach to the review phase as an opportunity for learning. The thesis will mention the positives but also note weaknesses and trades-offs.

6.6 Summary

Testing and validation phase validates that the application functions as a modular academic prototype and has useful user flows, intelligible screens, and travel functionality. Also it highlights some critical sections for improvement; automation of tests, persistence of hotel and car bookings, actual payments.

Chapter 7
7 Results, Limitations, and Deployment Considerations

7.1 Achievement Against Objectives

The primary objectives of the project are achieved, and a tourism application written in Flutter that provides modules to browse destinations, view tours, choose a hotel, rent a car, support maps, simulate payments, save a profile, view booking history and provide user listings is presented. The modules are not disparate screens but instead provide relevant flows to support user' travels and use common services between screens.

The objective of demonstrating modularity is also achieved through the division of screens, services, models, widgets and utilities, which helps in the readability of the project and ensures that the project can be explained in terms of both a user-flow and code-structure at evaluation.

7.2 User-Oriented Benefits

A major benefit from the user's perspective is the consolidation of fragmented tour planning in one interface. A user can search destinations, examine options, proceed to make a booking and check past reservations without having to leave the app. The common user experience is perhaps the most tangible result of the effort.

This product can be directly useful in its context with Pakistani focused content, localized payment indications and recognizable local travel cases. Rather than just modeling after a universal travel product, the content is geared toward a domestic experience.

7.3 Technical Contributions

From a technical perspective, the system demonstrates how a student developed mobile application can combine multi-platform UI development, cloud usage, local failure fall back, device permissions, image upload and module service layout in one transparent code structure. This provides a good reference case for a similar kind of project at university in a related area such as traveling services or information aggregation and services.

The idea of a hybrid storage solution used in some parts of the application can also be seen as a contribution. Where Firebase behavior is applicable and local fall back is used otherwise, the environment limitation common to academic demo and development environments is reflected.

7.4 Limitations and Constraints

The key limitations of the system, at this time, were simulated payment transaction and automated testing being only partially automated, session storage only for some booking types rather than persistent and the user inputted hotel and car entries being only stored locally instead of pushed to the server side.

However, all these are within reasonable scope for an academic project and are valuable should the system be thought for production. Documentation of these will help provide an honest profile as a robust prototype with an achievable path toward completeness.

7.5 Deployment Considerations

Stronger rules in Firebase would need to be implemented for production, the backend synchronization would need to be completed, a payment gateway would need to be integrated and all assets properly hosted, tests would need to be more thorough and final app store deployment requirements like performance profiling, inclusion of terms of service and release signing would have to be performed. Even with this, this project would scale in a coherent fashion to reach that production stage. The necessary conceptual structures for a production launch are all present: an identity management system, data models, a logical UI, and modularized services for each function, allowing this to be a both a fully deliverable academic project and an acceptable baseline for a product application.

7.6 Academic Value and Reusability

One of the final year projects most highly valued by employers and lecturers is one that can be justified on the day, as well as working, and it can also be understood and expanded upon. The application under development achieves this since domain mapping is easily apparent and the boundaries of modules are distinct, and it addresses a problem of direct societal relevance. The report, source code and placeholder figure strategy provide a system to communicate and present.

Another strong point is its usability as a starting point for the final year project, internship, research prototype, or start-up concept. Since it has already been modularized the existing work can be easily retained, refactored or enhanced and weaknesses may be readily targeted.

Chapter 8
8 Conclusion and Future Enhancements

8.1 Conclusion

The report has successfully described the research, designing, development and testing of the "Tours & Travel Mobile Application" project, which is a Flutter-based application designed for managing tourism that specifically focuses on the local traveling cases within Pakistan. The application has effectively proven that one well-designed mobile application is capable of holding the discovery of destination, booking systems, map, personal profile, simulated payment transactions, user-added places and a unified flow for all these actions together.

This final year software engineering product fulfills its criteria of being a significant work, both from a technical perspective and from the applicability within its domain, because the system has applied requirement engineering, modular design, user interface designing, data storage strategies, practical testing approaches to the actual and sensible issue, which have finally turned it into an application, feasible to demonstrate, stable to evaluate and expandable enough for further product-based developments.

8.2 Lessons Learned

A significant lesson from this project is that integration tends to be more difficult than individual page development. An individual page can appear complete, yet the actual benefits of a travel application is revealed when the authentication, selection, confirmation and history features are all functioning correctly and integrated. This reiterates the software engineering concept that the end user journey is just as important as the correctness of each component.

A further lesson is that pragmatic fallback design significantly improves academic software projects. With local and session-based support for features that would fail if fully server side dependent if given an inadequate server environment the project was demonstrably easier to test. This is not comparable to production engineering but significantly adds robustness to iterative development.

8.3 Future Enhancements

Further developments will include the ability to handle actual payment gateways, a recommendation engine, rating and reviews system, improved search capabilities,cloud storage for every booking category, an administration panel, push notifications and personalized user

experience based on analytics. The system could benefit from more sophisticated role models which handle services and moderators in a more direct way.

In addition to more functional work, testing needs to be carried out more systematically with automatic unit tests, service tests and integration tests as well as a more developed pattern for state management, more abstraction over routes and an enhanced level of security hardening. The purpose would be to develop the project more from an advanced academic prototype towards a system design ready for deployment.

8.4 Final Remarks

In brief, the project demonstrates the feasibility of applying mobile computing to achieve the digitization of tourism and the ways by which a student project can overcome a localized demand with a sound design. It is hoped that the ideas, architecture and documents in this thesis are helpful to assess and evolve the application.

9 REFERENCES

1. [1] Google, "Flutter documentation," Flutter. [Online]. Available: https://docs.flutter.dev/. [Accessed: 30 April 2026].
2. [2] Google, "Firebase documentation," Firebase. [Online]. Available: https://firebase.google.com/docs. [Accessed: 30 April 2026].
3. [3] Google, "Google Maps Platform documentation," Google for Developers. [Online]. Available: https://developers.google.com/maps/documentation. [Accessed: 30 April 2026].
4. [4] Google, "Material Design 3 guidelines," Material Design. [Online]. Available: https://m3.material.io/. [Accessed: 30 April 2026].
5. [5] Dart, "Dart language tour," Dart documentation. [Online]. Available: https://dart.dev/guides/language/language-tour. [Accessed: 30 April 2026].
6. [6] I. Sommerville, Software Engineering. London, U.K.: Pearson.
7. [7] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach. New York, NY, USA: McGraw-Hill.
8. [8] J. Nielsen, Usability Engineering. San Francisco, CA, USA: Morgan Kaufmann.
9. [9] OWASP Foundation, "OWASP mobile application security resources." [Online]. Available: https://owasp.org/. [Accessed: 30 April 2026].
10. [10] Baseflow, "Geolocator package documentation," pub.dev. [Online]. Available: https://pub.dev/packages/geolocator. [Accessed: 30 April 2026].
11. [11] Flutter Dev Team, "Google Maps Flutter package documentation," pub.dev. [Online]. Available: https://pub.dev/packages/google_maps_flutter. [Accessed: 30 April 2026].
12. [12] Flutter Dev Team, "Shared Preferences package documentation," pub.dev. [Online]. Available: https://pub.dev/packages/shared_preferences. [Accessed: 30 April 2026].
13. [13] Firebase, "Firebase Auth package documentation," pub.dev. [Online]. Available: https://pub.dev/packages/firebase_auth. [Accessed: 30 April 2026].
14. [14] Firebase, "Cloud Firestore package documentation," pub.dev. [Online]. Available: https://pub.dev/packages/cloud_firestore. [Accessed: 30 April 2026].
15. [15] Dart and Flutter package ecosystem, "pub.dev package repository." [Online]. Available: https://pub.dev/. [Accessed: 30 April 2026].